

Solid Modeling

3

CHAPTER OUTLINE

3.1 Introduction.....	126
3.2 Basics of solid modeling.....	127
3.2.1 Wireframe models.....	127
3.2.2 Surface models.....	129
3.2.3 Solid models.....	130
3.2.4 Major modeling schemes.....	132
3.2.4.1 Constructive solid geometry.....	132
3.2.4.2 Boundary representation.....	135
3.3 Feature-based parametric solid modeling.....	139
3.3.1 Geometric features.....	140
3.3.2 Sketch profiles.....	142
3.3.2.1 Sketch relations.....	142
3.3.2.2 Variational modeling.....	144
3.3.3 Parent–child relationships.....	149
3.3.4 Parametric modeling.....	150
3.3.5 Solid modeling procedure in CAD.....	151
3.3.6 Direct modeling.....	155
3.3.7 Geometric modeling kernels.....	155
3.4 Solid model build plan.....	157
3.5 Commercial CAD systems.....	160
3.5.1 General purpose codes.....	161
3.5.2 Special codes.....	161
3.6 Summary.....	162
Appendix A Sketch relations.....	162
Questions and exercises.....	164
References.....	167

With the basic understanding of geometric modeling discussed in Chapter 2, we are moving closer to the core of product design modeling—that is, solid modeling, especially feature-based parametric solid modeling, which is the key topic to be discussed in this chapter. In recent decades, the term *solid modeling* has been associated with the technology of using computer-aided design (CAD) systems to create the shape and form of part geometry and associated physical properties with a computer for the purpose of engineering designs. Today, CAD models with built-in essential product design information play a central role in e-Design.

Solid modeling in CAD applications has evolved through a series of phases in order to improve the geometric representation of physical artifacts or design concepts that are being developed in the engineering design process. It started in early 1960s when the first wireframe computer graphic was invented at the Massachusetts Institute of Technology (MIT) Lincoln Laboratory. In the mean time, design automated by computer (DAC-1), the first production interactive graphics manufacturing system, was developed by General Motors. Since then, with further development, surface modeling became a reality in the 1960s and solid modeling began in the 1970s, followed by feature modeling in the mid-1980s and parametric modeling by Parametric Technology in the late 1980s. With the development of feature modeling and parametric modeling in the 1980s, feature-based parametric solid modeling has since become the mainstream CAD theory in support of engineering design. It is well recognized that the most significant development appeared in the mid-1990s, in which major CAD tools were made available in personal computers (PCs) that allowed end users in mid- and small-size companies to be able to bring designs from the drawing board into digital form. More recently, direct modeling technology brought CAD one step further in support of engineering design by allowing designers to directly manipulate solid models by pulling or squeezing solid features on the computer screen using a mouse.

Today, major CAD systems employ feature-based parametric modeling techniques to support engineering designs through respective interactive user interfaces. Because solid modeling is the heart and soul of CAD, we devote this chapter to introducing basic knowledge in solid modeling methods and theory. This is the knowledge that readers must have in order to proceed with the study of the e-Design paradigm and gain practical skills in practicing e-Design, in which product geometry is represented in CAD solid models throughout the product development process.

This chapter is organized with the assumption that the reader has used CAD software (e.g., SolidWorks, Pro/ENGINEER) for creating solid models but has no or little background in solid modeling theory. If you are not familiar with CAD software, you are strongly encouraged to review excellent references for tutorial lessons, such as [Toogood and Zecher \(2012\)](#) for Pro/ENGINEER or [Planchard and Planchard \(2013\)](#) for SolidWorks. With the assumption that you are familiar with CAD software, we offer discussion on numerous topics involved in solid modeling, with examples extracted mostly from SolidWorks and Pro/ENGINEER.

Overall, the objectives of this chapter are (1) to provide an introduction to the basic solid modeling theories that help readers understand how the product design is realized in CAD, and (2) to help readers become familiar with the behind-the-scenes operations in CAD modeling so as to effectively use these tools for design. We also provide a short discussion on commercial CAD software tools, with the hope of offering readers guidance on selecting proper tools that are suitable for their specific needs.

3.1 Introduction

In the 1970s, nearly every engineering drawing produced in the world was done with pencil or ink on paper. A drafter leaned on the drawing board and used a T-square ruler, protractors, a compass, and templates to carefully sketch the lines, arcs, letters, and symbols that constitute an engineering drawing. Any changes or mistakes required erasing and redrawing, whereas major changes often necessitated recreation of the drawing from scratch. In manually created drawings, one of the most challenging tasks is that a drafter must envision the intersection of solid entities, unwrap the intersecting curves, and sketch the curves accurately on the drawing paper. Engineering drawing has been the backbone of product design and development for many years ([Bozdoc, 2003](#)).

Although engineering drawing still plays an important role in product design and manufacturing in many industrial sectors around the world, manual sketching for creating drawings has been gradually replaced by CAD (computer-aided design) software using computers. Beginning in the 1980s, CAD software reduced the need for draftsmen significantly, especially in small to mid-sized companies. The software's affordability and ability to run on personal computers in the mid-1990s allowed engineers to do their own drafting and analytic work to some extent.

In fact, instead of just creating drawings, CAD has fundamentally changed the way design is done. As in the manual drafting of technical and engineering drawings, the output of CAD conveys information, such as materials, processes, dimensions, and tolerances, according to application-specific conventions in solid models. Instead of drafting in digital form, designers use CAD to create product models in solid model forms with adequate product data, then they create drafting if necessary. CAD solid models offer flexibility and efficiency when making design changes; provide geometric and physical data that support product performance evaluations using computer-aided engineering (CAE); support virtual manufacturing, prototyping, manufacturing process planning, and product cost estimating; and offer product life cycle and product knowledge repository for archiving. Most important, product model in CAD serves as the centerpiece for e-Design.

The backbone of CAD is solid modeling. It is indispensable for designers to acquire adequate knowledge in CAD and solid modeling in order to effectively practice e-Design in support of engineering design. We introduce numerous theories and schemes that support product (or more specifically, parts) representation in solid models, with a focus on feature-based parametric solid modeling, which is the mainstream solid modeling method offered in major CAD systems. The main theme of the chapter is understanding the behind-the-scenes operations while you are using CAD for creating solid models. It is also important for readers to understand how CAD rebuilds solid models when a design change is made by changing dimension values associated with solid features.

We start in [Section 3.2](#) by introducing the basic theories of solid modeling, including constructive solid geometry (CSG) and boundary representation (B-rep), which are the two most widely used schemes for solid modeling. With a basic understanding of solid modeling, we discuss the main topic of the chapter in [Section 3.3](#)—that is, the feature-based parametric solid modeling method. In [Section 3.4](#), we offer the practical aspects of creating solid models by discussing model construction plans. We then provide a short overview of commercial CAD software in [Section 3.5](#).

3.2 Basics of solid modeling

Before getting into the main topic of this chapter—feature-based parametric solid modeling—we discuss a few important basic topics in solid modeling in this section. We start by discussing three basic methods for representing solid models: wireframe, surface, and solid forms. We include the advantages and disadvantages of each form, as well as the use of the models represented in the form for design and manufacturing applications. We will then narrow our focus to solid modeling, for which we introduce two major modeling methods: CSG and B-rep.

3.2.1 Wireframe models

Wireframe is the simplest and the earliest form of representing physical objects; it was first introduced in 1963 at MIT's Lincoln Laboratory. The wireframe form represents a shape by its characteristic

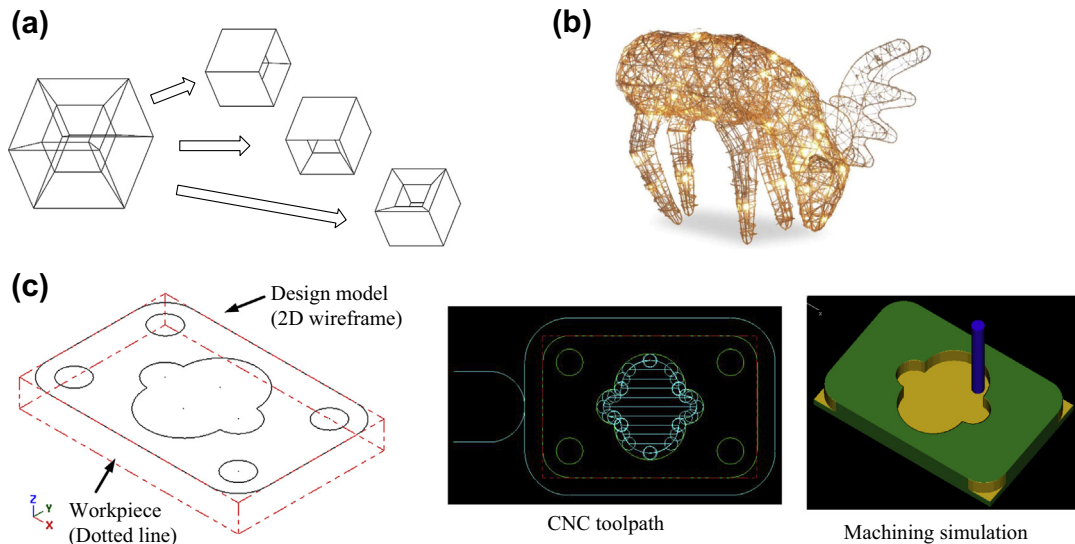


FIGURE 3.1 Wireframe Models. (a) Ambiguity in Representing a Solid Object. (b) Outdoor Christmas Decoration of a Rattan Reindeer (courtesy of <http://www.brookstone.com/pre-lit-outdoor-christmas-decorations-rattan-reindeer>). (c) Supporting Toolpath Generation in MasterCAM.

curves (lines, arcs, splines, and so on) and points, as illustrated in Figure 3.1a. A reindeer frame decoration, as shown in Figure 3.1b, which is displayed in residential front yards during Christmas, is a good example of wireframe model in real-world applications. The major advantages of this method are that it requires simple input from users and the modeling software is relatively easy to implement. One of the examples in its applications in design and manufacturing is the two-dimensional (2D) wireframe models in MasterCAM that supports numerical control (NC) toolpath generation for machining simple prismatic features, such as pocket milling or profile milling (Figure 3.1c), in which all contours exist in flat planes and only planar geometric information is required.

Although there are some applications that a wireframe model is able to support, there are major issues involved in representing solid models in wireframe. First, it is ambiguous for representing a solid object in a wireframe, as illustrated in Figure 3.1a, due to its inability to determine the inside or outside of a solid object. Second, a wireframe is not able to represent objects with nonpolygonal boundaries due to a lack of curvature information on surfaces. In addition, it is impossible to calculate mass properties of a solid object represented in a wireframe form. A wireframe model is not capable of supporting a finite element mesh for structural analysis of a physical object other than beam or truss structures. Generating a toolpath on a nonpolygon surface of a solid model represented in a wireframe is impossible due to its lack of surface geometric information.

Because of these reasons, no CAD tool uses wireframe alone to represent part geometry. Wireframes are only used in CAD as one of the options for visualizing solid objects—that is, the wireframe mode—due to its quick response in displaying objects on the computer screen without rendering.

3.2.2 Surface models

A surface can be thought of as an infinitely thin shell stretched over a wireframe. In addition to lines and points, surface models represent a shape by its surface geometry, as illustrated in Figure 3.2. A surface model includes information about the faces and edges of a part. Modeling methods for a surface offered in CAD include protrusions, such as extrude, sweep, loft, and revolve; interpolating points; and knitting and trim, as shown in Figure 3.2. To represent a solid object, the surfaces must form an airtight cavity that replicates the geometry of the object without any gap between them or any dangling surface or line. A hot air balloon shown in Figure 3.2f, is a good example of a surface model in real-world applications. Surfaces are commonly used to model complex, freeform (or organic) shapes that are commonly found in applications in the automotive, aircraft, mold, and consumer goods industries.

A surface model is good for visualizing complex surfaces and supports NC toolpath generation. As illustrated in Figure 3.3a, a toolpath was generated on a Coons patch in MasterCAM. The surface model is also widely used in finite element analysis (FEA) for thin-shell structures, as shown in Figure 3.3b, in which an FEA was carried out for a surface model that represents the mid-plane of a thin-shell solid object created in CAD. In addition, a stereolithography (STL) model, which is a surface model consisting of triangular facets that form an airtight cavity representing a solid object, is the de-facto standard for three-dimensional (3D) printing (also called rapid prototyping or solid freeform fabrication). An example of the STL model is given in Figure 3.3c.

As for modeling solid objects, the surface model generally works well. However, the mass or volume information of a solid object represented in a surface model is hard to determine, partly

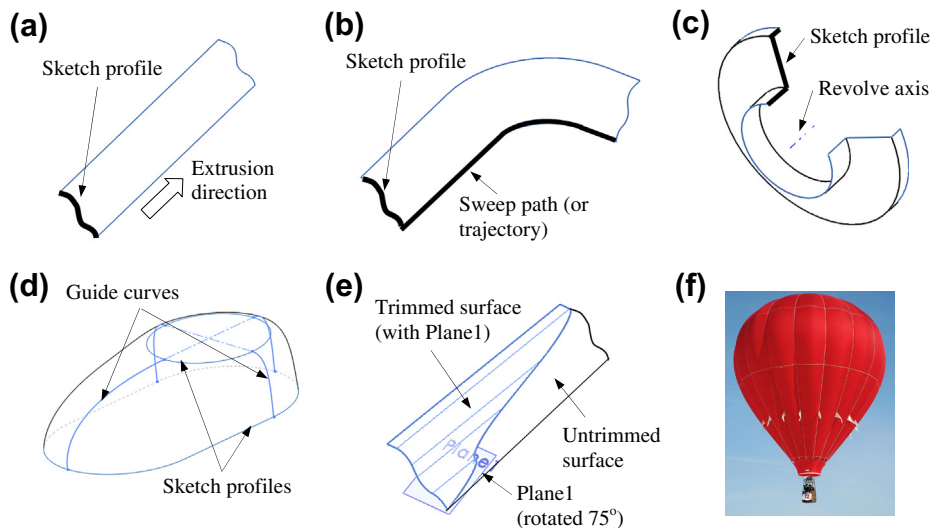


FIGURE 3.2 Creating Surface Models in Computer-aided Design. (a) Extrusion. (b) Sweep. (c) Revolve. (d) Loft (e) Trimmed. (f) A Hot Air Balloon.

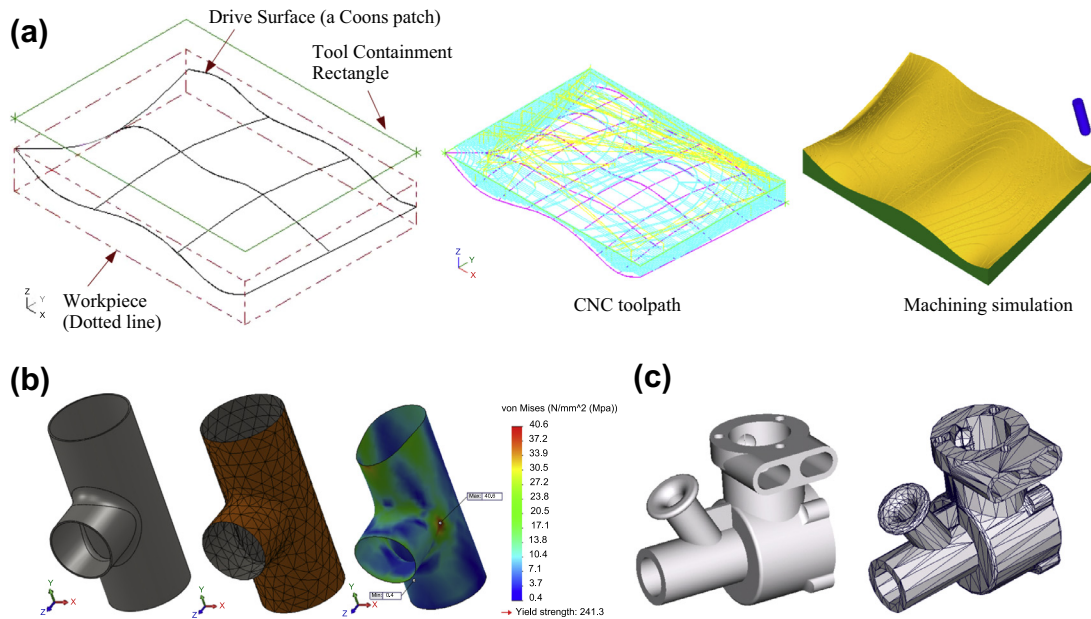


FIGURE 3.3 Surface Models for Support of Design and Manufacturing. (a) Toolpath and Machining Simulation in MasterCAM. (b) FEA of a Thin-shell Structure as a Solid Model in CAD (Left), Surface Model of Mid-plane (Middle) with Finite Element Mesh, and FEA Stress Fringe Plot (Right). (c) Engine Block as a Solid Model (Left) and Stereolithography (STL) Model (Right).

because a surface model lacks the mathematic representation of the solid object. Additional information must be added to a surface model in order to specify in/out and top/bottom of the physical object that the surface model represents.

3.2.3 Solid models

Solid models contain information about the edges, faces, and the interior of the part. The mathematical description contains information that determines whether any location is inside, outside, or on the boundary surface. Modeling a solid object in the solid model form generally includes primitive creation and Boolean operations, surface operations, protrusion operations, pick-and-place, feature-based modeling, and parametric modeling. It is important to note that individual CAD systems only use some of these methods for the modeling capabilities they respectively offer.

Primitives are basic solid objects with simple mathematical surfaces, as depicted in Figure 3.4a. These primitives can be controlled by a small number of parameters and positioned using a transformation matrix (as discussed in Chapter 2). Boolean operations, such as union, intersection, and difference, are used to make more complicated objects by combining the basic objects. This method is often referred to as CSG. More about this approach will be discussed in Section 3.2.4.

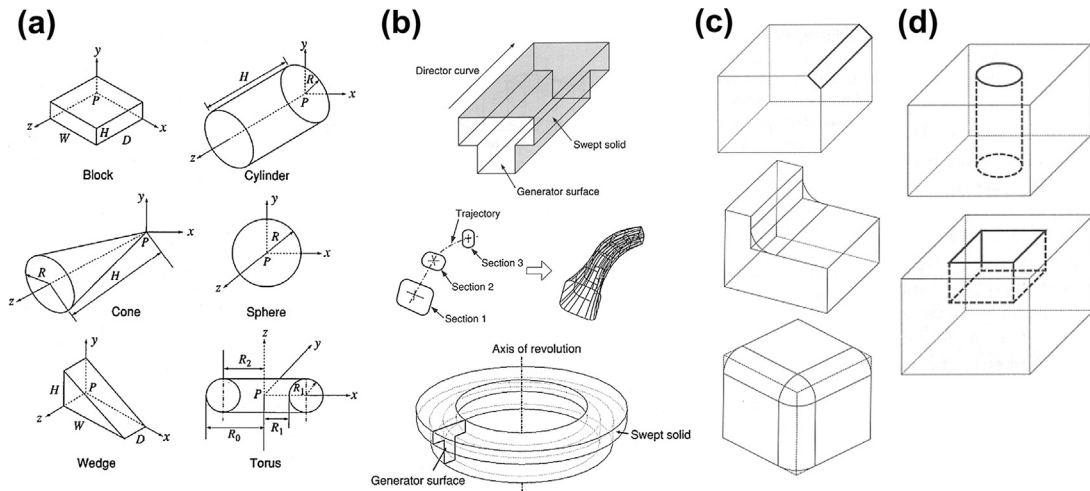


FIGURE 3.4 Solid Model Construction Methods. (a) Primitives and Boolean Operations. (b) Protrusion Operations. (c) Pick-and-place. (d) Feature Operations.

Surface operations trim and knit surfaces to form an airtight cavity that represents a solid object. This method is called boundary representation or B-rep. More details of this method are discussed in [Section 3.2.4](#).

Protrusion operations use 2D sketch profiles to generate a 3D solid by extruding, revolving, sweeping, and loft. Examples are shown in [Figure 3.4b](#).

Pick-and-place operates directly on the solid model surfaces, edges, and vertices to create a desired modification. Some examples include chamfering, rounding/filleting, drafting, and shelling, as illustrated in [Figure 3.4c](#).

Feature modeling mainly supports manufacturing operations. Manufacturing features are shapes having engineering manufacturing significance. They usually are the geometric embodiment of machining operations, such as hole, pocket, slot, and boss, as illustrated in [Figure 3.4d](#). Note that *feature* (instead of *manufacturing feature*) is a generic term used by CAD users and developers to refer to almost all kinds of geometric entities in solid modeling, sometimes including nongeometric entities, such as datum features (including planes, axis, points, coordinate systems).

Parametric modeling manipulates parameters to control the geometric shape of a solid object. Parameters come from dimensions in 2D profiles in sketch, dimensions on 3D solid features, and variables in user-defined equations. If defined properly, the entire part geometry can be controlled by a small number of key parameters. Design intents can therefore be captured through the change of the small set of parameters. Parametric modeling supports design parameterization; therefore, it becomes an indispensable part of the product design modeling in the context of e-Design. More about feature-based parametric modeling is discussed in [Section 3.3](#) as a key section of the chapter. Design parameterization, which is an important topic of this book, is discussed in Chapter 5.

A solid model is the ultimate way to represent general objects, which are physically solid objects. Solid models support NC toolpath generation of complex surfaces and meshing with solid elements for

finite element analysis. In addition, solid models are adequate for the calculation of mass properties in support of motion simulations. Solid models also support collision and interference checking, which are critical in assembly and kinematic analyses.

3.2.4 Major modeling schemes

As mentioned, there are several methods for constructing solid models. From a CAD user's perspective, protrusion and pick-and-place methods are most often employed. In terms of mathematically representing a solid object, two major modeling methods, CSG and B-rep, are widely employed by geometric modeling kernels, which are the core of CAD systems. More about kernels is discussed in [Section 3.2.5](#). In this subsection, we discuss these two modeling methods in greater detail to offer readers a more in-depth understanding of CAD theory and behind-the-scenes mathematic operations.

3.2.4.1 Constructive solid geometry

CSG is a modeling method that supports the construction of solid objects through operations on solid primitives. CSG records both the information of operations and information of the primitives. The major components of the CSG method are primitives and instances, as well as the Boolean set operations and CSG tree.

3.2.4.1.1 Primitives and instances

A typical CSG system uses primitives, such as cylinders, boxes, cones, spheres, as shown in [Figure 3.4a](#), as well as their instances, as the building blocks for constructing solid models. The idea is to use the primitives that can be manipulated easily in the computer system. Each individual primitive is stored as a geometric family together with a set of parameters. An instance of the primitive family is a scaled and/or transformed replica of its original. The scaling may be uniform or differential. It is a method of keeping primitives in their basic minimum condition, such as unit cube and unit sphere. For example, a cube belongs to the family of box with all three parameters—length, width, and height—having the same value.

3.2.4.1.2 Boolean set operations and CSG tree

To construct a complex object, the CSG approach decomposes it as a compound object composed of a number of primitives. CSG typically uses Boolean set operations, including union, difference, and intersection, to construct objects. These are mathematical operations taken from set theory. The primitives involved in an operation are referred to as operands.

A union of primitives gives a volume occupied by each operand minus the volume shared (or overlapped) by them, as illustrated in [Figure 3.5b](#), in which union operation is carried out for two primitives P and Q ([Figure 3.5a](#)). In CAD, such a union operation is realized in many ways. For example, in [Figure 3.5b](#), you may sketch a profile and extrude it for Q, and extrude (both sides) a square block from Q for P. Implicitly, union operation is employed in CAD.

Difference operations require two operands playing different roles. The base operand defines the source volume (Q in [Figure 3.5c](#)) and the second operand defines the volume to be removed (P in [Figure 3.5c](#)). The resulting object contains the volume of the base operand but not the second operand. For example, the object shown in [Figure 3.5c](#) represents difference operation $Q - P$. In CAD, you may

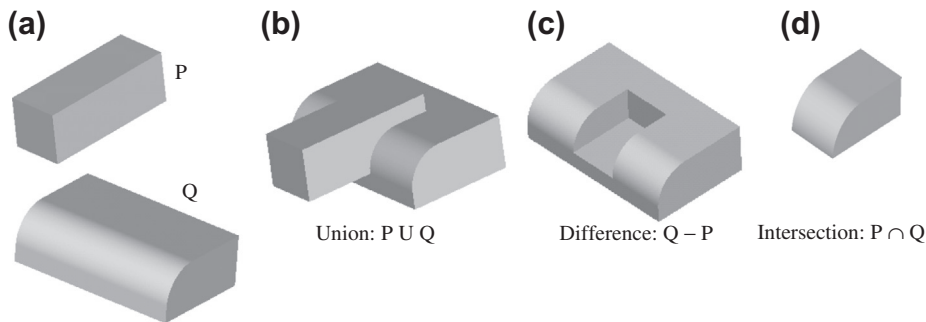


FIGURE 3.5 Boolean Operations. (a) Solid Primitives P and Q. (b) Union of P and Q. (c) Difference of P and Q. (d) Intersection of P and Q.

create a sketch profile and extrude it for Q, and extrude a square blind cut P to Q to yield the object shown in Figure 3.5c.

An intersection of primitives gives a volume common to all operands, for example, the object shown in Figure 3.5d representing an intersection $P \cap Q$. Note that there is no direct intersection operation in major CAD systems.

In fact, as a user, we do not see any direct Boolean operation capabilities offered by CAD systems. However, there are capabilities for users to cut and union features as illustrated in Figure 3.5. In general, location and orientation of primitives involved in the Boolean operations determine largely the resulting object. A primitive or its instance is usually scaled, translated, and rotated to a prescribed location and orientation before carrying out a Boolean operation.

As seen clearly, Boolean operations are binary, involving two (or two sets of) primitives. The primitives involved in each operation and the sequence of operations create a so-called CSG tree. A CSG tree, as shown in Figure 3.6 schematically, is a binary tree with leaf nodes as the primitives and interior nodes (or branch) represent Boolean set operations. The root node represents the final part.

The CSG tree creates a procedural model that specifies how the solid features are combined to form the final solid model. In general, the solid model must be “evaluated” by computing intersecting curves

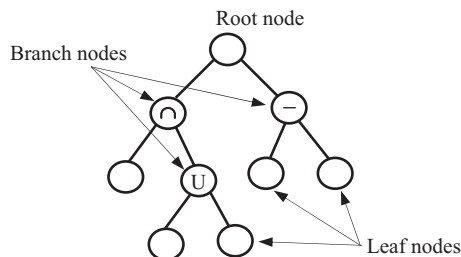


FIGURE 3.6 A Schematic of a Constructive Solid Geometry Tree.

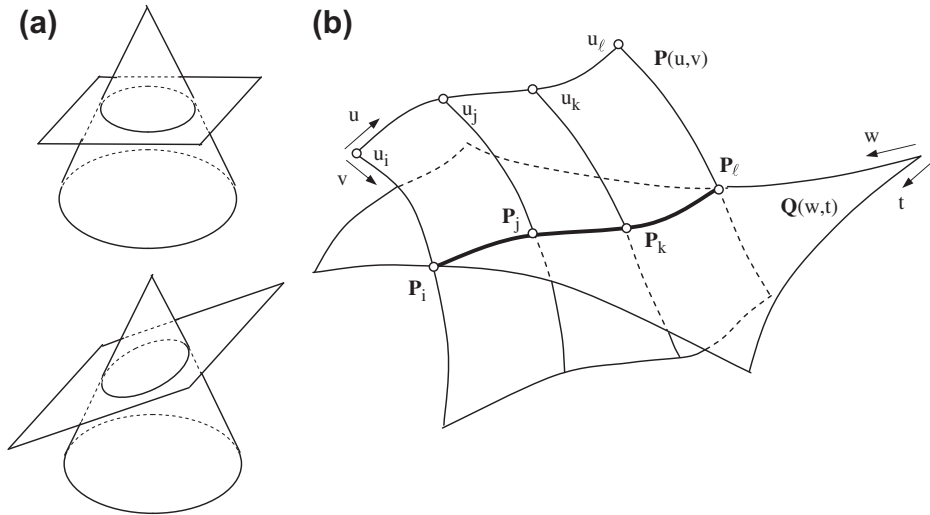


FIGURE 3.7 Surface to Surface Intersection. (a) Intersecting Two Standard Primitives. (b) Intersecting Two Parametric Surfaces.

from the parametric surface equations of the geometric features, based on the position and orientation of the primitives. Quantitative information, such as intersecting curves, must be generated and stored to define and to display the solid model.

The curve intersecting two surfaces can be analytical only for basic primitives, such as a circular cone intersecting with a plane. Conic curves, such as a circle or ellipse, are generated as a result of the intersection, as illustrated in Figure 3.7a, which can be represented analytically. However, analytical representations of intersecting curves are generally not available, so approximation methods must be used. We discuss one approximation method that calculates intersecting curves of two parametric surfaces as an example.

Consider two parametric surfaces $P(u,v) = [P_x(u,v), P_y(u,v), P_z(u,v)]$ and $Q(w,t) = [Q_x(w,t), Q_y(w,t), Q_z(w,t)]$, where u and v , and w and t are parametric coordinates of the two surfaces, respectively (see Figure 3.7b). The intersecting curve is constructed by generating parametric curve that passes through a number of intersection points.

The intersecting curve must satisfy the following equation:

$$P(u,v) - Q(w,t) = 0. \quad (3.1)$$

There are four unknowns u , v , w , and t , but only three equations (parametric equations of Eqn (3.1) in the x -, y -, and z -directions, respectively). Basically, the intersecting curve cannot be obtained by solving Eqn (3.1).

One approach to construct the intersecting curve:

1. Fix a value of, for example, the u parameter of the surface $P(u,v)$, to generate a curve on the surface (i.e., $P(u_j,v)$), as shown in Figure 3.7b.

2. Compute the intersection points of this curve with the surface $Q(w, t)$ by solving numerically.

$$P(u_j, v) - Q(w, t) = 0 \quad (3.2)$$

We have now three equations (parametric equations of Eqn (3.2) in the x-, y-, and z-directions, respectively), and three unknowns (v, w, and t).

3. Repeat Steps 1 and 2 for as many u_j as needed. For example, if a cubic curve is desired, then four intersecting points at $u = 0, 1/3, 2/3$, and 1 can be calculated for a cubic spline curve to approximate the intersecting curve, as illustrated in Figure 3.7b.

As can be seen from this discussion, CSG is intuitive because the concept of solid model construction is in some sense parallel to manufacturing operations. The solid model constructed is always valid (except if one unions two cones only at their vertices, for example). Another advantage of the CSG is that the solid model requires a small data set (i.e., the primitives involved and CSG tree). In general, the CSG model is an unevaluated model, which must be “evaluated” for numerous purposes (e.g., display on computer screen) and calculating engineering data (e.g., mass properties). As discussed, evaluating a CSG model can be inefficient because the process involves computations, such as calculations of intersecting curves.

3.2.4.2 Boundary representation

B-rep is an important method of 3D modeling for solid objects. A B-rep model represents a solid object by assembling (or gluing) surfaces to form an “airtight” boundary that encloses the 3D space occupied by the object, as illustrated in Figure 3.8.

In B-rep, a solid model is bounded by faces, a face is bounded by edges, and an edge is bounded by vertices. Essentially, there is a hierarchy of four levels of geometric entities: volume, face, edge, and vertex. Face, edge, and vertex are topology entities that specify connectivity information. In addition to topological entities, the corresponding geometric entities are surface, curve, and point that define the shape, location, and orientation of the entities. Both geometric and topology data must be defined to construct the solid model. Both must be stored in a database.

For example, as shown in Figure 3.9, a block is defined in a volume with its six boundary faces. Each rectangular face is defined by its own edges to form an enclosed surface. All the surfaces are

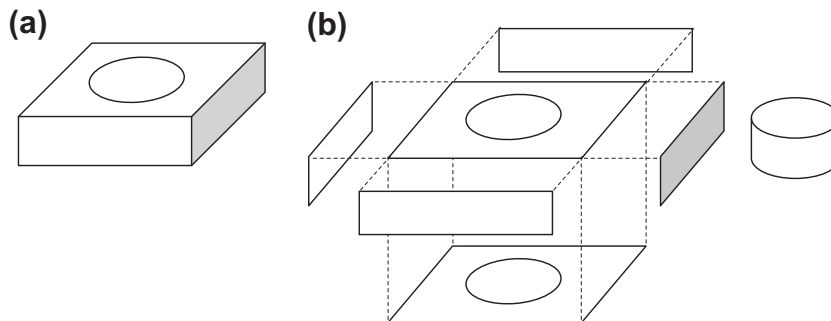


FIGURE 3.8 A Boundary Representation Model of a Solid Object. (a) Solid Model. (b) Boundary Surfaces.

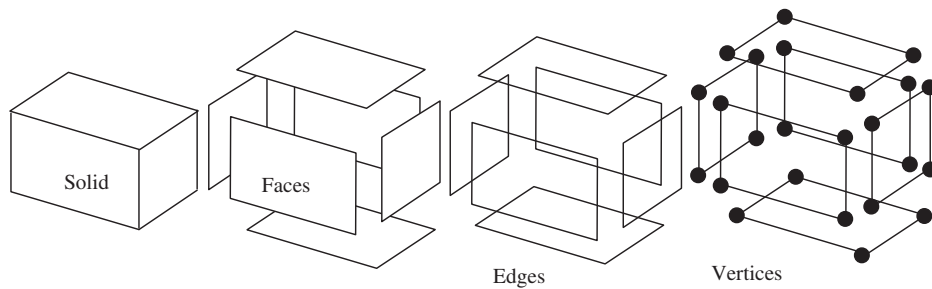


FIGURE 3.9 The Boundary Representation Model of a Rectangular Block.

joined along the common edges of their respective neighboring faces. Every edge—a straight line in this case—is bounded by its end points.

One advantage of the B-rep model is that the model is fully evaluated; that is, all geometric entities are explicitly defined and are ready for display. However, a complex B-rep model requires a relatively large database. It is less intuitive to create solid models using the B-rep method as compared with CSG because users must deal with points, curves, and surfaces instead of primitives that are much more relevant to physical objects. More importantly, a B-rep model constructed by a designer may be invalid, and a B-rep model must be verified for its topology before putting it to use.

How can we (or the computer, in this case) tell if a B-rep model is topologically valid? In a topologically valid model, all faces are properly “glued” to wrap the solid object airtight, all edges are properly “joined” that fence the face, and all edges are properly bounded by end vertices. In addition, there must be no dangling faces or edges, and no split solid object.

Which objects in [Figure 3.10](#) are topologically valid? Apparently, objects in [Figure 3.10a](#) and [b](#) are valid, but not [Figure 3.10c](#). It is apparent that normal objects found in nature have the property that, at every point on the boundary, a small enough sphere around the point is divided into two pieces: one inside and one outside the object. This property can be easily verified for the objects in [Figure 3.10a](#) and [b](#). The so-called nonmanifold models break this rule. For example, in the object shown in [Figure 3.10c](#), a small sphere around any point on the four edges of the rectangle at the bottom face of the cube is divided into four pieces: two inside and two outside. Essentially, the object in [Figure 3.10c](#) is two solids “welded” along the edges of the rectangle on the bottom face. The welded edges are

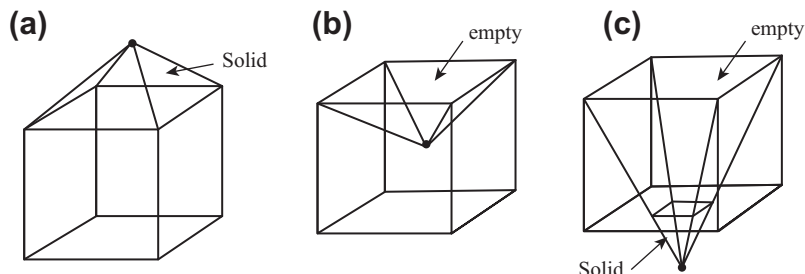


FIGURE 3.10 Examples of Manifold (a, b) and Nonmanifold (c) Objects.

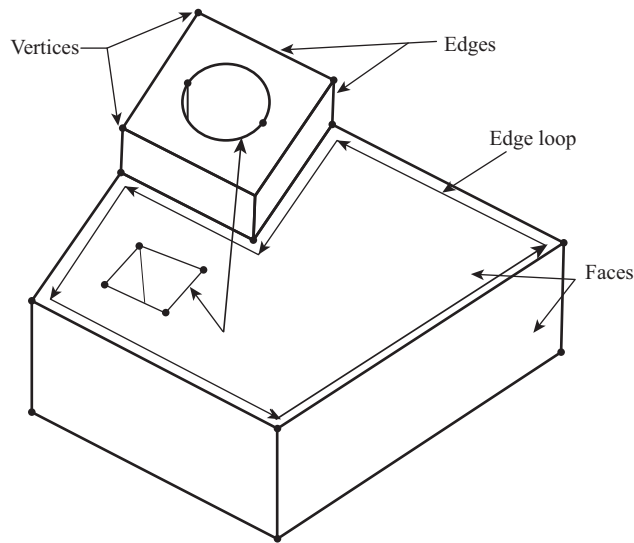


FIGURE 3.11 Illustration for the Topological Entities Employed in the Euler-Poincare Law.

infinitely thin without a cross-sectional area, which is physically impossible. A nonmanifold object, such as the one on [Figure 3.10c](#), is considered to be topologically invalid.

To ensure the topological validity of a B-rep model, the number of its topological entities must satisfy the Euler-Poincare law, which is stated as follows:

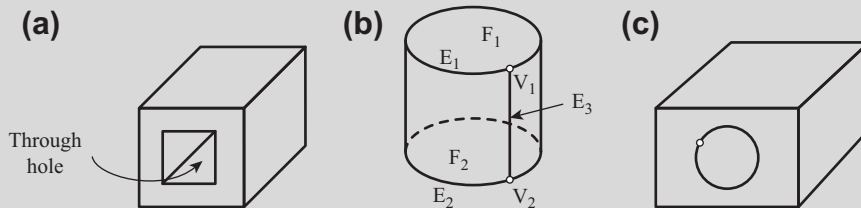
$$v - e + f = 2(s - h) + r \quad (3.3)$$

where v , e , f , s , h , and r are the numbers of vertices, edges, faces, solids, through holes, and rings, respectively. Note that r can be a ring or an inner loop of edges that are completely within a face. [Figure 3.11](#) offers an illustration for the topological entities mentioned in [Eqn \(3.3\)](#).

The following simple examples illustrate the law and verify the topological validity of the respective physical objects.

EXAMPLE 3.1

Use the Euler-Poincare law to verify if the following objects are topologically valid: (a) a rectangular block with a rectangular through hole, (b) a circular cylinder, and (c) a rectangular block with a circular through hole.



Continued

EXAMPLE 3.1—CONT'D**Solutions**

For part (a), we have $f = 10$ (four exterior, four interior, and two ends), $v = 16$, $e = 24$, $s = 1$, $h = 1$, and $r = 2$. From [Eqn \(3.3\)](#), we have

$$v - e + f = 16 - 24 + 10 = 2$$

and

$$2(s - h) + r = 2(1 - 1) + 2 = 2.$$

Therefore, part (a) is topologically valid. Physically, the block is a valid object.

For part (b), we have $f = 3$, $v = 2$, $e = 3$, and $s = 1$. Note that a silhouette edge must be added (and the associated vertices) to a cylindrical surface when counting the number of topological entities. From [Eqn \(3.3\)](#), we have

$$v - e + f = 2 - 3 + 3 = 2$$

and

$$2(s - h) + r = 2(1 - 0) + 0 = 2.$$

Therefore, part (b) is topologically valid.

We are applying the same principle for part (c)—that is, adding a silhouette edge to the circular cylindrical surface inside the rectangular block.

Hence, for part (c), we have $f = 6 + 1 = 7$, $v = 8 + 2 = 10$, $e = 12 + 3 = 15$, $s = 1$, $h = 1$, and $r = 2$. From [Eqn \(3.3\)](#), we have

$$v - e + f = 10 - 15 + 7 = 2$$

and

$$2(s - h) + r = 2(1 - 1) + 2 = 2.$$

Therefore, part (c) is topologically valid.

Boundary representation is essentially a local representation connecting faces, edges, and vertices. An extension of this is to group the primitive geometric entities of the shape into logical units called geometric features. Features are the basis of many other developments, allowing high-level “geometric reasoning” about shape for comparison, process planning, manufacturing, etc. Feature-based modeling is discussed next in [Section 3.3](#).

Compared to the CSG representation, which uses only primitive objects and Boolean operations to combine them, boundary representation is more flexible and has a much richer operation set. This makes boundary representation a more appropriate choice for CAD systems. CSG was used initially by several commercial systems because it was easier to implement. The advent of reliable commercial B-rep kernel systems, such as Parasolid and ACIS, has led to widespread adoption of B-rep for CAD. B-rep kernels systems offer CAD-like operations, such as protrusion, chamfer, blending, drafting, shelling, tweaking, and other operations. Note that most CAD systems employ both CSG and B-rep for solid modeling, or at least the major principles of these methods. In general, CSG keeps the relationship between features, and B-rep stores topological and geometric data for display and computations. More about geometric modeling kernels can be seen in [Section 3.3.7](#).

3.3 Feature-based parametric solid modeling

Most modern CAD software employs a methodology called feature-based parametric solid modeling as the major interface for users to interactively create solid models. Feature-based modeling approach is more desirable in constructing solid models, in which designers use features that correspond to physical entities to construct solid models, instead of dealing with primitive geometric entities, such as points, curves, and solid primitives. The features available in CAD are usually designed to relate to how engineers think in their design and manufacturing work. The parametric modeling method allows designers to create solid models in such a way that by varying a few parameters (e.g., geometric dimensions), the solid models rebuild automatically as intended (i.e., capturing design intent). For example, a hole in the block shown in Figure 3.12a is intended to stay at the middle of the block when the width of the block changes. To capture this design intent, first the sketch profile of the base block must be fully defined (Figure 3.12b), with dimension d2 as a design variable that is to vary. The hole must be placed to the profile of the base block with its width dimension (d1 in Figure 3.12c) related to that of block width d2, as $d1 = 0.5d2$. This one-way parameter assignment is essentially parametric modeling.

In most CAD, the base block is a protrusion feature generated by extruding the sketch profile shown in Figure 3.12b along an extrusion direction that is normal to the sketch profile. The hole is an extrude cut feature that does not require users to sketch its profile, which is called a pick-and-place feature.

The question is how CAD software captures our design intent. How does CAD handle the sketch profile of the block when you make a change? How does CAD rebuild the part and what are the potential pitfalls you, as a designer, should avoid?

In this section, we intend to answer these questions. We start by revisiting “features” to have a better understanding of the terminology so that the “feature-based modeling” becomes more vivid. We then discuss the variational modeling method that CAD employs to support the needed calculation for determining a sketch profile. We discuss parent–child relationships, which are generated

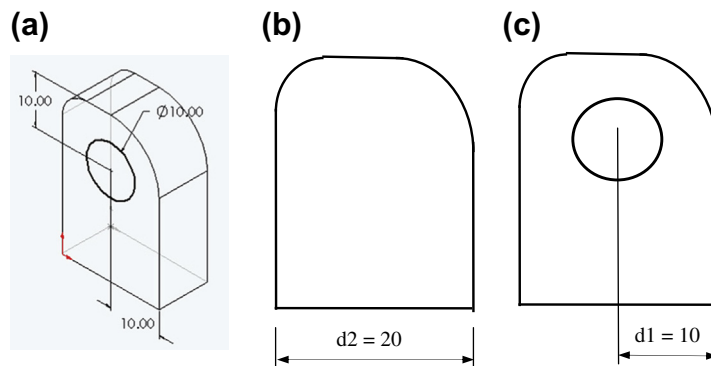


FIGURE 3.12 (a) The Block Example for the Illustration of Design Intent Capturing Using Feature-based Parametric Solid Modeling Method. (b) Sketch Profile of the Base Block with Width Dimension d2 Shown. (c) Position of the Hole by Dimension d1, Which can be Parametrically Related to Dimension d2.

when features are added to the solid model. Finally, we discuss the parametric modeling method and the solid modeling procedure we often exercise in CAD, in which we show how CAD rebuilds a part by walking through the steps using a simple example. We also discuss a newly developed modeling method called direct modeling. This section is wrapped up by a short introduction to geometric modeling kernels.

3.3.1 Geometric features

The term “feature” implies different meanings in different engineering disciplines. This has resulted in many ambiguous definitions for feature. A feature in computer-aided design (CAD) usually refers to a region of a part with certain geometric or topological properties (Pratt and Wilson, 1985). These are more precisely called geometric (or form) features. Geometric features contain both shape information and parametric information of a region of interest. They are now ubiquitous in most current CAD software, where they are used as the primary means of creating 3D solid models.

Another frequently used feature is the manufacturing feature, which can be defined simply as a geometric shape and its manufacturing information to create the shape. Manufacturing features support the generation of process plans in a feature-based process planning system. Machining features are an important subset of manufacturing features. A machining feature can be regarded as the volume swept by a “cutting” tool, which is always a negative (subtracted) volume. Some CAM software, such as CAMWorks, offers an automatic (manufacturing) feature recognition capability that recognizes manufacturing features embedded in CAD solid models and generates a toolpath accordingly. There are also tolerance features that specify deviations from the nominal form (or shape), size, or locations, such as surface flatness, circularity, and concentricity. Finally, there is the concept of assembly feature, which encodes the assembly method between connected components.

The features mentioned previously are highly related to part geometry. There are also non-geometric features, such as material features that specify material composition and heat treatment for a part.

In this subsection, we discuss geometric features from a design perspective. Instead of providing a generic and philosophical discussion, we narrow the focus to CAD solid modeling. Nowadays, almost everything that contributes to the construction of a solid model in CAD is called a feature. Basically, geometric features involved in creating a solid object can be categorized into the five groups: construction (or datum) features, shape (or protrusion) features, pick-and-place (or hard-coded) features, mirror and pattern features, and thickened features, as shown in Figure 3.13. Therefore, features can be thought of as the individual shapes that, when combined, make up the part.

Construction or datum features—such as coordinate systems, planes, axes, or points—are auxiliary entities that aid solid model creation. Default construction features, including a coordinate system and

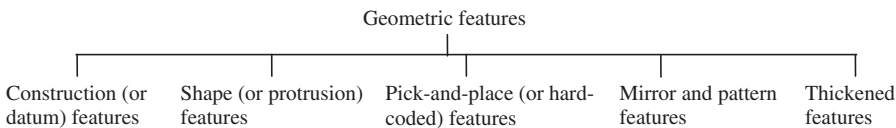


FIGURE 3.13 Classification of Geometric Features.

three perpendicular planes (front, top, and right), are provided in CAD as starting point for part solid modeling (and assembly).

Protrusion features (some also call them “sweep” features) are the most important set of features that support solid modeling. Such features include extrude, sweep, loft (or blend), and revolve, in which sketch profiles are required, as illustrated in Figure 3.14. In addition to protrusion that adds volume, protrusion can also be used to create cut features that remove volume from existing objects. Attributes, such as protrusion direction (one or both sides), are options offered to designers to complete a protrusion feature conveniently.

Pick-and-place features are hard-coded features, including chamfer, fillets, rounds, draft, and holes, which are placed on a face or an edge of existing objects without sketching a profile. Such features are often added in the final stage of the solid modeling process.

Mirror and pattern features are created from existing features, as illustrated in Figure 3.15. Mirror copies the selected features or all features, mirroring them about the selected plane or face (Figure 3.15c). Pattern, as shown in Figures 3.15a and b, repeats the selected features in an array based on a seed feature. The array can be linear (a linear pattern), a circular (a circular pattern), or following a curve. Some CAD, such as SolidWorks, offers feature copy and paste capabilities, in which designers can pick an existing feature (e.g., a through hole), then copy and paste it on a different face of the solid object in a different orientation.

The thickened feature creates a solid feature by thickening one or more adjacent surfaces. For example, the tracked vehicle roadarm surface model discussed in Chapter 2 (Section 2.7.2) was created in B-spline surfaces, imported into SolidWorks, and then thickened for a solid model in support of structural analysis using FEA. Note that except for the construction features, geometric features are solid features.

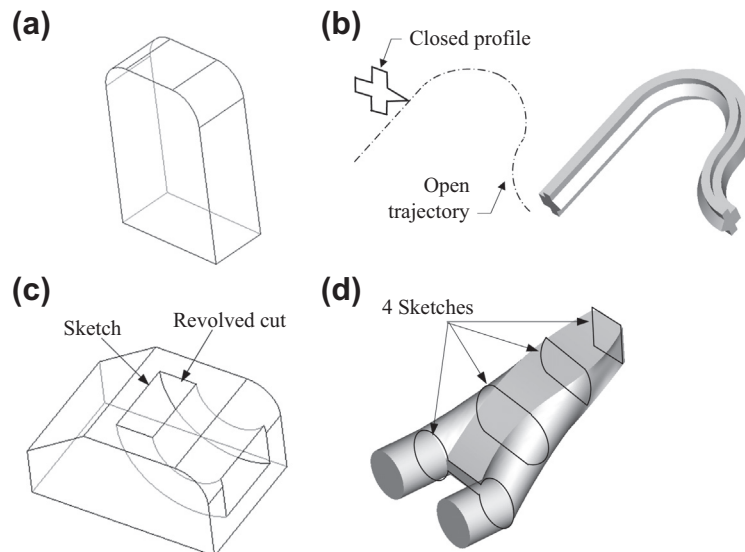


FIGURE 3.14 Protrusion Features. (a) Extrusion Feature. (b) Sweep Feature. (c) Revolved Cut Feature. (d) Blend Feature.

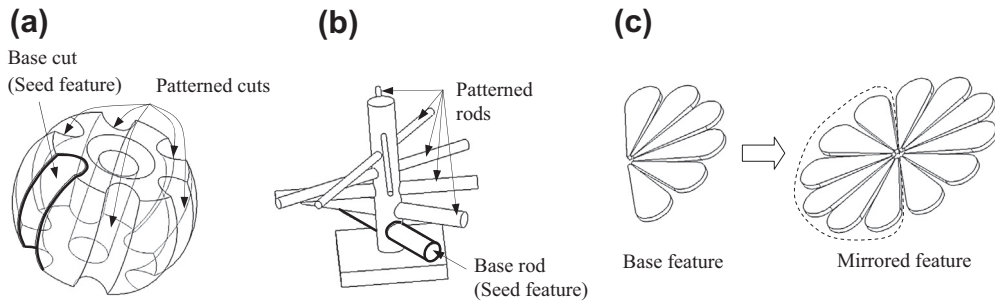


FIGURE 3.15 Copy, Pattern, and Mirror Features. (a) Patterned Cut Features. (b) Patterned Extrusion Features. (c) Mirrored Feature.

Among these features, protrusions are probably the most important ones in supporting designers to create most features in solid modeling. Protrusions features (including cuts) require designers to sketch a profile, in which a section view of the feature is defined. How does CAD support designers to create sketch profiles interactively? How does CAD receive inputs from the designer, then formulate and solve equations to define the sketch profile mathematically? We discuss sketch profiles in the next subsection.

3.3.2 Sketch profiles

When we start a new part, we pick a sketch plane and create a profile. The profile is the basis for a 3D model. We usually create a profile on one of the default construction planes (front, top, and right), or a created plane. In sketching a profile, as a CAD user, we create an open or closed profile with lines, arcs, and so on. A CAD system, such as SolidWorks or Pro/ENGINEER, automatically adds sketch relations (also called sketch constraints) to relate or constrain entities; for example, a straight line connects to a circular arc with a tangent relation at the junction point. After completing the profile, we add dimensions and enter proper values to adjust the profile that meets our design requirements. After entering or modifying a dimension value, CAD is able to adjust the profile as a logical consequence of the change. How does the CAD system do that? CAD employs the so-called variational modeling theory in sketch mode.

3.3.2.1 Sketch relations

In this section, we use examples to illustrate the sketch relations and variational modeling technique. We assume SolidWorks sketch mode to be more specific. Other CAD systems follow a similar approach.

We often start the sketch profile at the origin; that is, we use the origin as the anchor for the profile. SolidWorks (and other modern CAD systems) creates relations for the geometric entities in the profile, based on how these entities are created. Designers add dimensions (or more relations) to make the profile fully defined (or fully constrained).

In SolidWorks, before creating any dimensions, the geometric entities, including lines and vertices, are either in black or (mostly) in blue color. As illustrated in [Figure 3.16](#) with a simple example, black

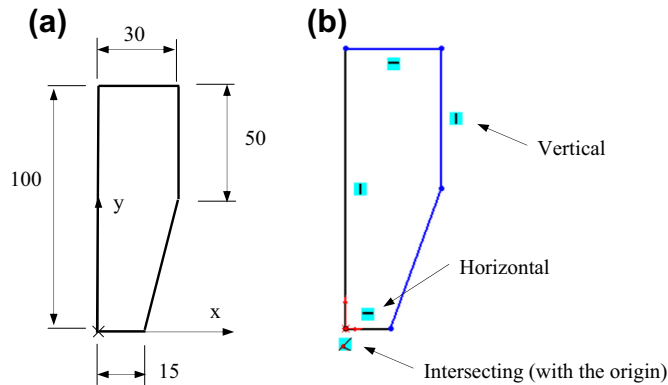


FIGURE 3.16 Sketch Profile as Designed (a) and with Underconstrained Relations (b).

indicates that the entity is fully defined. Blue indicates that the entity is not fully defined and is free to change in certain way. You may drag a vertex or line in blue color to see how it can be changed in SolidWorks.

When you add dimensions or relations, affected entities will change from blue to black color, indicating they become fully defined as a result. When all the entities of the sketch are in black, the entire sketch is fully defined. In the model tree (called Browser in SolidWorks), the (—) sign in front of the sketch is removed. Sometimes, the sketch is overconstrained when a conflict occurs or more dimensions (or relations) are created than required.

Note that some CAD systems, such as Pro/ENGINEER, offer “smart” sketching tools. When turned on, design intent is inferred, and sketch relations and dimensions are added automatically to make the sketch profile fully defined. In some cases, line and curve entities are slightly adjusted with imposed relations. For examples, two straight lines that are nearly perpendicular may “snap” perpendicular with a perpendicularity relation (Figure 3.17a), and two fillets with about the same radii may be adjusted to have equal radii with an equal radii relation added (Figure 3.17b). For more details on the sketch relations in SolidWorks and Pro/ENGINEER, please refer to Appendix A.

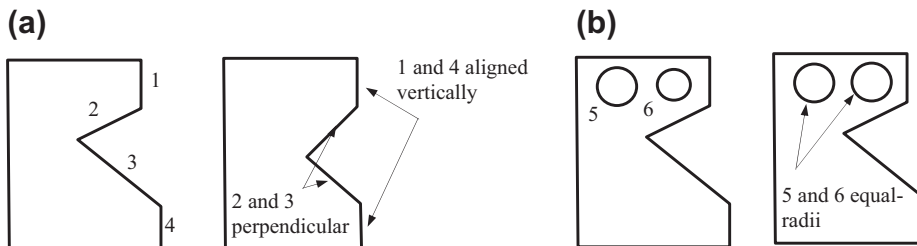


FIGURE 3.17 Effects of Imposed Sketch Relations to the Profile. (a) Perpendicular and Alignment Relations were Added. (b) Equal-radii Relation was Added.

The question is how SolidWorks knows what dimensions and relations are just right for the sketch (i.e., to make it fully defined). How does SolidWorks figure out if your sketch is overdefined or underdefined?

3.3.2.2 Variational modeling

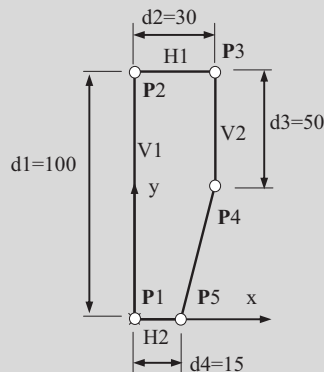
CAD software employs the variational modeling theory in sketch mode. The first step involved in variational modeling is identifying a set of characteristic points (or vertices) on the sketch profile. For example, the sketch shown in Figure 3.16 has five characteristics points that are at the corners of the polygon. Then, system of equations are derived that incorporate the relations and dimensions defined on the sketch to relate the x- and y-locations of these characteristic points. The system of equations are solved to determine the locations of the characteristic points.

In formulating the system of equations, the number of equations must be identical to the number of unknowns and equations must be linearly independent for a unique solution. This is when a sketch is called fully defined. When the dimension values are changed, the same system of equations is solved again for the locations of the characteristic points.

When the number of equations is greater than the number of unknowns, we have an overdefined sketch. When the number of equations is less than the number of unknowns, we have an underdefined case, where a (—) sign will stay in front of the sketch in the Browser.

EXAMPLE 3.2

Determine if the following sketch profile with the relations and dimensions is fully defined. In this sketch, **P1** is fixed to the origin, and there are four relations (two horizontal and two vertical) and four dimensions. If we change the dimension d3 from 50 to 100 and d4 from 15 to 30, what will happen to the profile? Would CAD accept such changes and be able to regenerate the profile? If we add a dimension, such as the length of the line segment **P4P5**, would the sketch profile still be fully defined?



Solutions

There are five characteristic points in the sketch profile, **P1**, **P2**, **P3**, **P4**, and **P5**; therefore, we have $5 \times 2 = 10$ unknowns and we need to have 10 linearly independent equations to solve for the unknowns. The following equations are derived from the relations and dimensions defined on the sketch.

EXAMPLE 3.2—CONT'D

Because point **P1** coincides with the origin, we have the following two equations:

$$\text{Coincident: } P1x = 0 \quad (1)$$

and

$$\text{Coincident: } P1y = 0 \quad (2)$$

The remaining equations are:

$$V1: P2x - P1x = 0 \quad (3)$$

$$d1: P2y - P1y = d1 \quad (4)$$

$$H1: P3y - P2y = 0 \quad (5)$$

$$d2: P3x - P2x = d2 \quad (6)$$

$$V2: P4x - P3x = 0 \quad (7)$$

$$d3: P3y - P4y = d3 \quad (8)$$

$$H2: P5y - P1y = 0 \quad (9)$$

$$d4: P5x - P1x = d4 \quad (10)$$

These ten equations are linearly independent. How can you tell? You may arrange these ten equations into a matrix form and check the rank of the matrix.

If we change the dimension $d3$ from 50 to 100 and $d4$ from 15 to 30, points **P4** and **P5** coincide, making the length of the line segment **P4P5** zero. Most CAD software, such as SolidWorks, will not accept any line or curve entity with zero length.

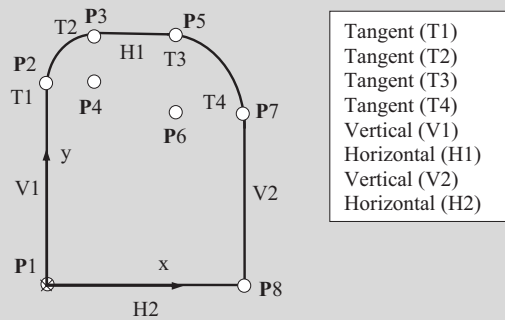
Because we have already ten linearly independent equations that solve uniquely for the ten unknowns, adding more dimensions, such as the length dimension for the line segment **P4P5**, causes the profile to become overdefined.

Now, let us take a look at a bit more complex problem, in which the profile consists of circular arcs.

EXAMPLE 3.3

Add adequate dimensions to the following sketch profile with the given relations to make the profile fully defined. In this sketch, **P1** is fixed to the origin, and there are another eight relations, as shown below. Formulate system of equations and solve them for the locations of the characteristics points that determine the shape of the profile.

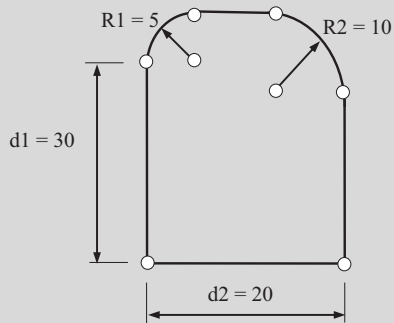
Continued

EXAMPLE 3.3—CONT'D

Tangent (T1)
Tangent (T2)
Tangent (T3)
Tangent (T4)
Vertical (V1)
Horizontal (H1)
Vertical (V2)
Horizontal (H2)

Solutions

There are eight characteristic points (including the arc centers); therefore, there are $8 \times 2 = 16$ unknowns, and we need to have 16 linearly independent equations to solve for the unknowns. The following equations are derived from the relations given to the profile shown above.



and

$$\text{Fix: } P1x = 0, \quad (1)$$

$$\text{Fix: } P1y = 0 \quad (2)$$

$$V1: P2x - P1x = 0 \quad (3)$$

$$T1 \text{ (and } V1): P4y - P2y = 0 \quad (4)$$

$$T2: P3x - P4x = 0 \quad (5)$$

$$H1: P5y - P3y = 0 \quad (6)$$

$$T3 \text{ (and } H1): P5x - P6x = 0 \quad (7)$$

EXAMPLE 3.3—CONT'D

$$T4: P7y - P6y = 0 \quad (8)$$

$$V2: P7x - P8x = 0 \quad (9)$$

$$H2: P1y - P8y = 0 \quad (10)$$

We need six more equations. These six additional equations come from the dimensions we are about to add to the profile.

If you add the four dimensions d1, d2, R1, and R2, as shown on previous page, the profile becomes fully defined. Why? Let us take a look at the equations that the dimensions will provide.

$$d1: P2y - P1y = d1 \quad (11)$$

$$R1(T1 \text{ and } T2): P4x - P2x = R1 \quad (12)$$

$$P3y - P4y = R1 \quad (13)$$

$$R2(T1 \text{ and } T2): P5y - P6y = R2 \quad (14)$$

$$P7x - P6x = R2 \quad (15)$$

$$d2: P8x - P1x = d2 \quad (16)$$

Now we have all 16 equations identified. Among them, [Eqns 1–3, and 10](#) are trivial; therefore, they are removed together with the four unknowns (i.e., $P1x = P1y = P2x = P8y = 0$).

We assemble the remaining 12 equations for the 12 unknowns in a matrix form as follows.

$$\begin{array}{l}
 (11) \\
 (5) \\
 (13) \\
 (12) \\
 (4) \\
 (7) \\
 (6) \\
 (15) \\
 (14) \\
 (9) \\
 (8) \\
 (16)
 \end{array}
 \begin{bmatrix}
 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \\
 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0
 \end{bmatrix}
 \begin{bmatrix}
 P2y \\
 P3x \\
 P3y \\
 P4x \\
 P4y \\
 P5x \\
 P5y \\
 P6x \\
 P6y \\
 P7x \\
 P7y \\
 P8x
 \end{bmatrix}
 =
 \begin{bmatrix}
 d1 \\
 0 \\
 R1 \\
 R1 \\
 0 \\
 0 \\
 0 \\
 -R2 \\
 -R2 \\
 0 \\
 0 \\
 d2
 \end{bmatrix}
 =
 \begin{bmatrix}
 30 \\
 0 \\
 5 \\
 5 \\
 0 \\
 0 \\
 0 \\
 -10 \\
 -10 \\
 0 \\
 0 \\
 20
 \end{bmatrix}$$

The system of equation can be solved using, for example, Matlab. We are solving five cases: Case 1: d1 = 30, R1 = 5, R2 = 10, d2 = 20 (base case); Case 2: d1 = 60, R1 = 5, R2 = 10, d2 = 20 (taller profile); Case 3: d1 = 30, R1 = 5, R2 = 5, d2 = 20 (equal fillet radii); Case 4: d1 = 30, R1 = 5, R2 = 15, d2 = 20 (zero length profile); and Case 5: d1 = 30, R1 = 5, R2 = 25, d2 = 20 (penetrating profile).

The Matlab script for solving the equations is shown (next page) is, followed by the resulting profiles.

Continued

EXAMPLE 3.3—CONT'D

EDU>>a =

1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	-1	0	0	0	0	0	0	0	0	0
0	0	1	0	-1	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0	0
-1	0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	-1	0	0	0	0	0
0	0	-1	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	-1	0	0	0
0	0	0	0	0	0	-1	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	-1	0
0	0	0	0	0	0	0	0	-1	0	1	0	0
0	0	0	0	0	0	0	0	0	0	0	1	0

Case 1	
EDU>>c=[	30
	0
	5
	5
	0
	0
	0
	-10
	-10
	0
	0
	20]

EDU>>b=inv(a)*c	
b =	
30	
5	
35	
5	
30	
10	
35	
10	
25	
20	
25	
20	

Case 2	
EDU>>c=[	60
	0
	5
	5
	0
	0
	0
	-10
	-10
	0
	0
	20]

EDU>>b=inv(a)*c	
b =	
60	
5	
65	
5	
60	
10	
65	
10	
55	
20	
55	
20	

Case 3	
EDU>>c=[	30
	0
	5
	5
	0
	0
	0
	-5
	-5
	0
	0
	20]

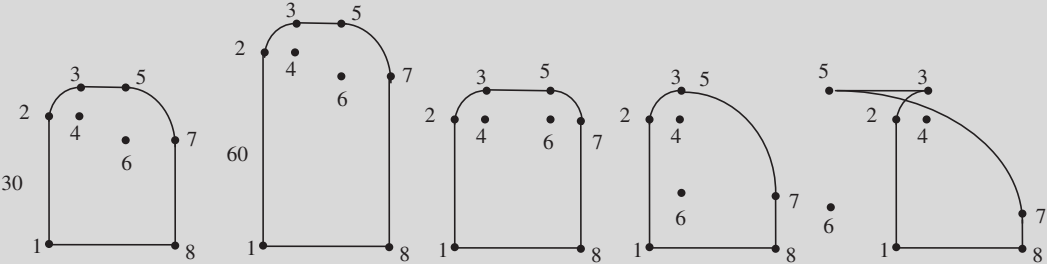
EDU>>b=inv(a)*c	
b =	
30	
5	
35	
5	
30	
15	
35	
15	
30	
20	
30	
20	

Case 4	
EDU>>c=[	30
	0
	5
	5
	0
	0
	0
	-15
	-15
	0
	0
	20]

EDU>>b=inv(a)*c	
b =	
30	
5	
35	
5	
30	
5	
35	
5	
20	
20	
20	
20	

Case 5	
EDU>>c=[	30
	0
	5
	5
	0
	0
	0
	-25
	-25
	0
	0
	20]

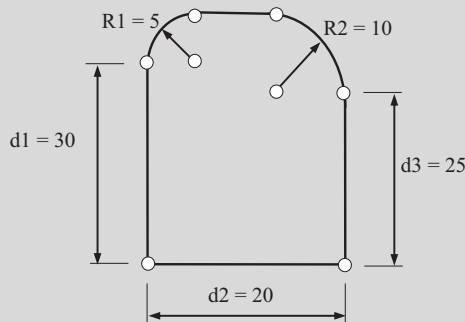
EDU>>b=inv(a)*c	
b =	
30	
5	
35	
5	
30	
-5	
35	
-5	
10	
20	
10	
20	



EXAMPLE 3.3—CONT'D

A number of important points are observed from Example 3.3:

1. Solutions exist for all five cases, which is because the number of equations and number of unknowns are identical, and the equations are linearly independent. As a result, the matrix equation is nonsingular and can be solved for a unique solution.
2. It is apparent that if we add a dimension d3 shown below, the sketch becomes overconstrained. Now, if we add d3 and remove d2, would the sketch be fully defined? The answer is no. Adding d3 creates a redundancy dimension because d3 is determined by $d1 + R1 - R2$. Removing the equations contributed by d2 and adding those contributed by d3 will make the system of equations linearly dependent; therefore, the resulting matrix equation becomes singular.



3. Cases 1–3 are regular profiles that CAD generates. For Case 4, because there is a zero-length line segment **P3P5**, some CAD programs, such as SolidWorks, prompt an error message and will not generate the profile. For Case 5, although the entities penetrate to each other, indicating a physically infeasible profile, some CAD systems, such as Pro/ENGINEER, check the validity of the profile and prompt with a warning message; however, others, such as SolidWorks, do not catch the problem and generate an invalid sketch profile anyway.
4. In both examples, we have all linear equations. In some cases, for instance, if angle dimensions are present, nonlinear equations are required. In these cases, an iterative numerical method, such as Newton's method, may be employed for solving the nonlinear equations.

3.3.3 Parent-child relationships

Once a sketch profile is completed, a solid feature can be created by using, for example, one of the protrusion capabilities. The first solid feature serves as the first building block, called the base feature, in the model construction process. Follow-on features are added to the base feature or existing solid features.

Depending on the sequence of feature construction, there is a parent-child relationship created between solid features. For example, the part shown in Figure 3.18a consists of five solid features. The first solid feature is the base block, which was created by extruding a sketch profile like that of Figure 3.12b along the extrusion direction perpendicular to the sketch plane of the profile. A center through hole was then added as a cut extrusion feature (or a pick-and-place feature), with a sketch

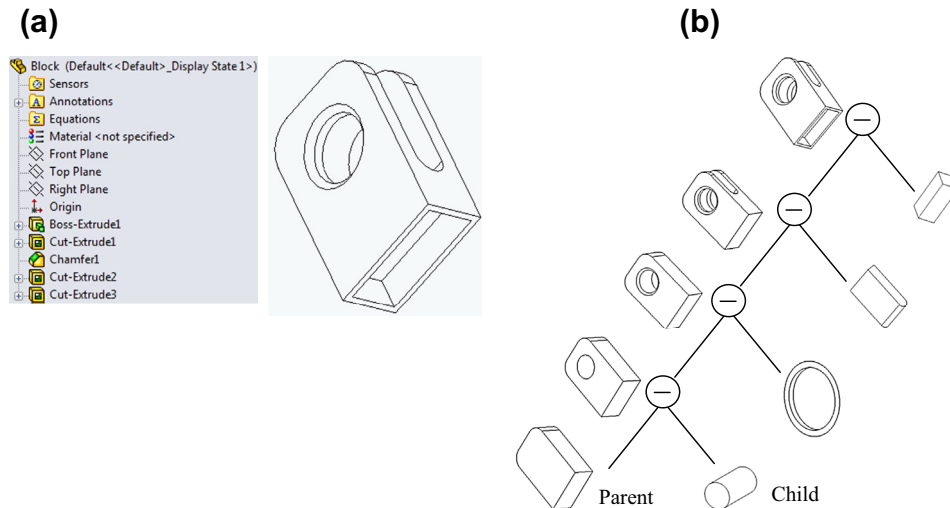


FIGURE 3.18 The Parent–child Relationships. (a) The Block Solid Model and Feature Tree in Browser. (b) Feature Construction Sequence in a Constructive Solid Geometry-like Tree.

placed on the front face of the base block, in which the hole is placed with position dimensions referred to the right and bottom edges of the base block, respectively. As a result, hole became a child feature of the base block, as illustrated in Figure 3.18b. The third feature is a chamfer (a pick-and-place feature) placed on the outside circle of the hole on the front face of the block. As a result, the chamfer is a child feature of the hole. The fourth and fifth features are the side cut and bottom slot, respectively, as shown in Figure 3.18b; both are child features of the base block.

In addition to parent–child relationships between solid features, a solid feature may be a child feature of a construction feature, and vice versa. For example, the base block of Figure 3.18b is a child feature of a default construction plane because its sketch profile was created on the plane.

Parent–child relationships are critical in the construction and rebuild of the solid model. Changes made to the parent feature will propagate to all child features during the rebuild. On some occasions, the part may not be rebuilt successfully due to numerous reasons, mainly because the part is not properly parameterized. Also, operations such as deleting, suppressing, or hiding a parent feature will affect all its child features. Therefore, it is extremely critical that designers arrange the sequence of feature construction and the way the child feature is related to its parent feature. A desirable solid model should have less coupled parent–child relationships between features. Solid model construction sequence or history is critical in the feature-based modeling approach. More about the design parameterization is discussed in Chapter 5.

3.3.4 Parametric modeling

Unlike the variational modeling technique that formulates and solves a system of equations, parametric modeling adopts a one-way assignment approach. For example, two dimensions d_0 and d_1 can be

parametrically related as $d0 = d1 \times 2$, in which $d0$ is a dependent parameter and $d1$ is an independent parameter that is free to change.

Such “assignment”-type equations are explicit and they are solved sequentially, in which each assigned value is computed as a function of previously assigned or computed values. Therefore, parametric modeling is also called “unidirectional” modeling or “procedural” modeling. In the above example, $d1$ must be defined first, and then $d0$ can be evaluated. The solid model must be rebuilt (regenerated) by propagating the changed parameter (dimension) through all equations that involve the parameter.

Computation in solving the explicit equation sequentially is efficient and straightforward. However, this approach lacks flexibility in relating parameters. For example, in the variational modeling method, $d0 = d1 \times 2$ can be written as $d0 - d1 \times 2 = 0$, in which either $d0$ or $d1$ can be independent.

3.3.5 Solid modeling procedure in CAD

After reviewing the solid modeling methods discussed so far, we revisit the general process of creating solid models in CAD. In the meantime, we walk through how CAD rebuilds the part when we make a design change, using a simple example. By going through this exercise, we hope you gain a better understanding of the behind-the-scenes operations while using CAD for solid modeling.

In general, as illustrated in [Figure 3.19](#), when we start a new solid model, we are given datum features, such as datum planes and datum coordinate systems. It is in general a good idea to develop a modeling plan before beginning the actual modeling work. More about modeling plans is discussed in [Section 3.4](#). At the beginning of creating a new part, we usually pick a datum plane and create a sketch profile for the first (or base) solid feature using one of the protrusion capabilities. As discussed before, a variational modeling technique is employed in CAD to determine the locations of characteristic points of the sketch profile. After the base solid feature is created, we either add more solid features by repeating the same process, sketch and make a cut feature, or place a pick-and-place feature on the existing features. We repeat some or all steps to create more features. In the meantime, CAD records the feature creation sequence in the model tree and parent–child relationships between features. Once a solid model is completely created, we often make a few adjustments to make sure the solid model accurately represents the design of the part. CAD rebuilds the part based on the changes we made by updating features (both datum and solid features) following the feature creation sequence, one feature at a time. For each feature, the CAD system does the following:

1. Takes the new dimension values, from user input and computation through parameter relations, to update the sketch profile first. In the sketch profile, a variational modeling technique is exercised. The system of equations that govern the profile of the section are solved again for the new parameter values.
2. Rebuilds the geometric features using the new sketch profile by
 - a. New parameter values in the protrusion direction, including extrude, revolve, sweep, or loft; and
 - b. Feature attributes, such as one side or both sides.
3. If the feature cannot be rebuilt (e.g., when an invalid geometric feature is encountered), an error message will appear. For example, in SolidWorks, a Rebuild Errors window appears, as shown in [Figure 3.20](#).

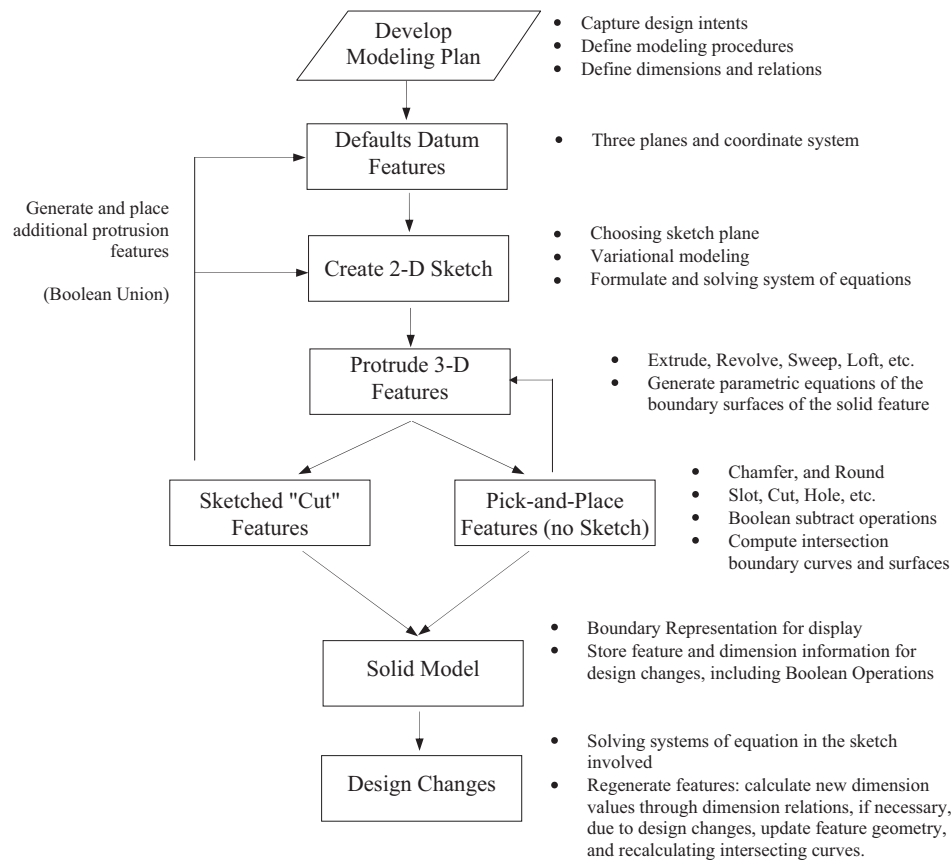


FIGURE 3.19 General Solid Model Creation Process.

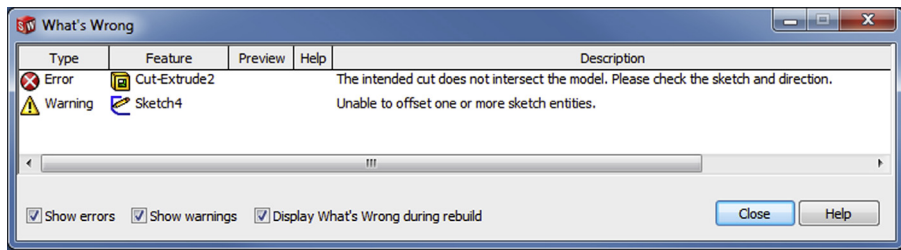


FIGURE 3.20 The Rebuild Error Message Window in SolidWorks.

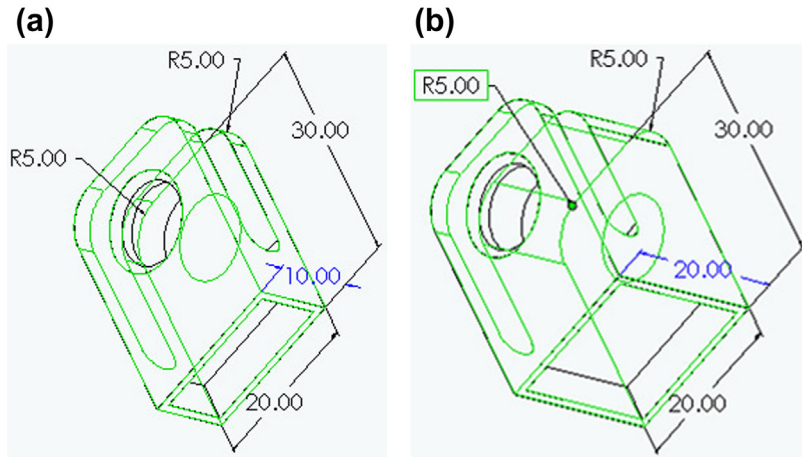


FIGURE 3.21 Change of the Depth Dimension of the Base Block from 10 to 20. (a) Solid Model before Design Change. (b) Solid Model after Design Change.

4. When this error message appears, we must read the messages carefully and identify which feature is problematic. Once the problematic feature is identified, it is a good practice to try to figure out what the problem was, and then undo the change to restore the dimension values, fix the problem, and try a similar change again. In SolidWorks or Pro/ENGINEER, you may choose *Edit > Undo Change Dim* (or CTL + Z) to undo the change.
5. Find child features through the parent-child relations. Note that while rebuilding child features, intersecting curves of feature boundary surfaces may need to be recomputed.
6. Repeat steps 1–5 until all the features are rebuilt.

In the following, we walk through the part rebuild process using the block example shown in Figure 3.18. We first change the depth dimension of the base block (d5) from 10 to 20, as shown in Figure 3.21. The regeneration follows the feature creation sequence, as shown in Figure 3.18b, which is described below.

1. First feature: base block

The sketch profile is unchanged. There is no need to resolve the system of equations. Therefore, the only action for CAD is to update the width of the base block on one side along the extrude direction, which is the attribute of the base block.

2. Second feature: big hole

Both placement data (placement plane, placement references, and dimensions) and hole dimension are unchanged. The hole is rebuilt following the feature attribute (i.e., through all and one side). The intersecting curve (in this case, the circle in the back face) is computed.

3. Third feature: side cut

Check sketch profile. Is the sketch profile changed? Why and why not? The cut feature is regenerated following the feature attribute (i.e., through all and cut directions), as

illustrated in Figure 3.22a. The intersecting curves of the base block and the cut features are computed.

4. Fourth feature: chamfer

This feature is not affected because the placement edges (the circle on the front face) and the size of the chamfer are unchanged.

5. Fifth feature: cut on the bottom face

Check the sketch profile. Is the sketch profile changed? Yes, because the rectangular profile is defined with an offset from the exterior rectangle, as illustrated in Figure 3.22b. The system of equations will be resolved for the four characteristic points (corner points of the rectangle) due to the change of d_5 . The cut feature is regenerated following the feature attribute—that is, blind with depth 5 and one side with the same cut direction. The intersecting curves are computed.

If we change the width of the base block (d_5) from 10 to 0.3, which feature(s) will fail to rebuild? Both the side cut and bottom cut features will not be generated because there is not enough room for the sketch profiles to be generated. However, CAD usually stops at the first unsuccessful feature; therefore, in this case, the rebuild error on the side cut will be reported.

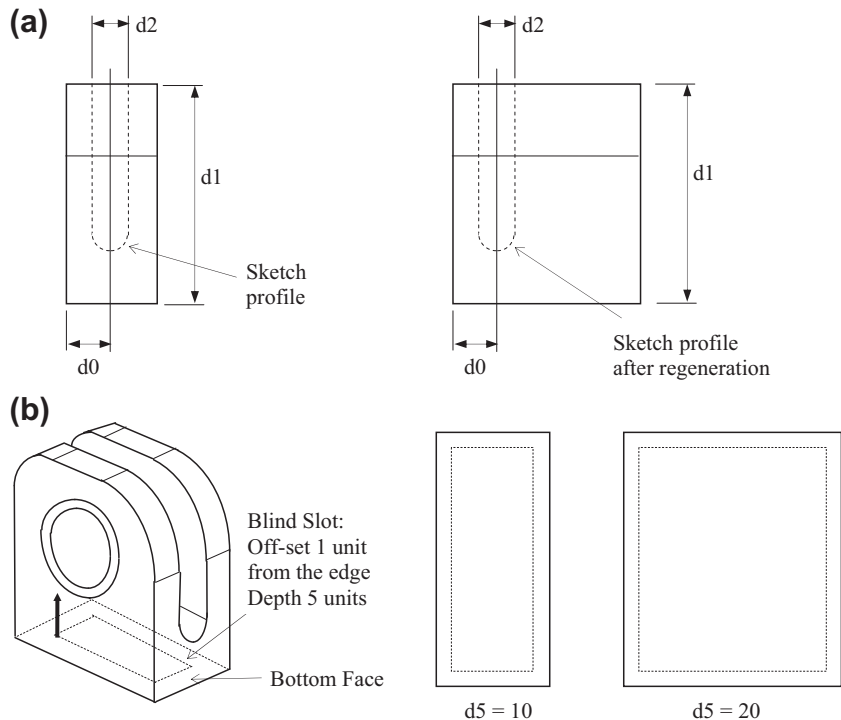


FIGURE 3.22 Feature Rebuilds. (a) Side Cut. (b) Bottom Cut.

3.3.6 Direct modeling

As can be seen from the discussion above, the parametric modeling approach requires the designer to anticipate design changes and accordingly define features, add relations to sketch entities, and add parameter relations between features. As a result, the solid model is created in such a way that a design modification (e.g., change in a dimension value) triggers rebuild in solid features in a prescribed manner. Feature-based parametric modeling is a structured modeling process, in which feature creation sequence or history tree masters the model rebuild process and design intent is captured implicitly through sketch relations, parent–child relationships, and parametric relations between dimensions.

Although the feature-based parametric modeling is indispensable in support of product design in the e-Design paradigm, capturing design intents in complex solid models is not always straightforward, to say the least. It requires the designer's effort, considerable planning, and careful implementation in achieving such parametric solid models. In general, parametric CAD tools lack ease of use, speed, and modeling flexibility. It requires a relatively steep learning curve and modeling effort upfront for the designer, and the solid models created suffer from model interoperability issues; that is, a CAD model created in software A cannot be understood or imported to software B with features and dimensions due to the nature of “history-based” model.

The newly developed direct modeling approach provides a geometric-based modeling strategy that gives designers the power to quickly define and edit geometry by simply clicking on the model geometry and moving it. Designers can focus on creating geometry rather than building features, adding constraints and design intent into their models and therefore speeding up design, saving time and development costs, and increasing productivity. The direct modeling paradigm is especially suited to the needs of designers working with legacy and heterogeneous CAD data. The direct modeling eliminates the need to access feature-level information to implement design changes. Designers can easily edit, modify, and repurpose solid models from any CAD sources.

Both Pro/ENGINEER (Creo™ 2.0 and higher) and SolidWorks (2012 and newer) are equipped with direct modeling (also called direct model editing) capabilities, which is built on top of existing feature-based parametric modeling technique. With the added direct modeling capability, designers are able to copy, move, split, replace, offset, push, and drag geometry to create the result as desired, instead of clicking on a dimension, entering a different value, and asking for model rebuild. In addition, with direct modeling capability, CAD automatically imports nonnative, imported model geometry without a model tree. The imported geometric model can be modified through direct geometry manipulation.

In general, parametric modeling is a history-based modeling method that enables design automation and creates product platforms for a product family, which are suitable for product design strategy that is aimed to be family-based or platform-driven. On the other hand, direct modeling is a geometry-centered and history-free approach that supports quick and easy 3D solid model construction, allows design change through direct manipulation of geometric models, and supports direct geometry-editing from any CAD source. There are pros and cons of these two methods. They are not exclusive but in general complement each other.

3.3.7 Geometric modeling kernels

No matter what kind of modeling method is offered by a CAD system, the core of any CAD software is its geometric modeling kernel. The kernel is key to support underline computing and modeling capabilities of solid objects, as well as output or export solid models, including 2D drawings, from 3D

geometry. All commercially available solid modeling systems today are built on top of a geometric modeling kernel (also referred to as a modeling engine or geometry library). This is the library of core mathematical functions that defines and stores 3D solid objects in response to users' commands. The kernel library processes commands input through the application's user interface, stores the results, and submits the output to the graphics package for display, as illustrated in [Figure 3.23](#).

Following [Figure 3.23](#), it is commonly understood that there are two layers of information created when designers work with CAD. The top layer records user interaction with the CAD through either feature-based or direct modeling capabilities in the form of geometric features, including sketches, attributes, parameters, and equations, and feature construction sequence or history tree. On the bottom layer is the resulting geometric entities or objects. A history-based CAD system is basically recording every function it sends to the kernel into the history tree. For example, a sketch with an extrusion distance creates an extruded feature. Constraints control the size and position of the new feature. Then, a Boolean function is added to specify whether the feature is added or removed from the parent geometry. The Boolean function with the feature and related parameters are passed to the kernel and the resulting geometry is processed and revealed. This kernel function is processed every time this feature is rebuilt. The kernel function along with its required parameters is very specific to the kernel as discussed above. It is highly unlikely that another kernel will understand this very specific function, and even if it did the geometrical results could be very different. This is one of the major issues to address in solid model interoperability among CAD systems.

Some geometric modeling kernels such as ACIS (Spatial Inc.), Parasolid (Unigraphics Solutions, Inc.), and SMLib (Solid Modeling Solutions) are licensed by their respective developers for use in many different CAD systems. Others, such as thinkkernel (think3), Granite One (Pro/ENGINEER), and UPG2 (Varimetrix Corporation), are proprietary kernels developed exclusively for a specific CAD system.

Apart from the underlying functionality supported, both licensed and proprietary kernels offer distinct advantages. CAD systems that license the same kernel can directly exchange model files that the kernel generates. For example, you can load SAT (ACIS) files directly into a CAD system that uses the ACIS kernel. On the other hand, developers who use different kernels in their CAD systems must write specific translators to read and write model files for import and export. Exporting and importing parametric solid models is not straightforward. More about CAD model translation is discussed in Chapter 6.

The geometric modeling kernels adopted by major commercial CAD systems are summarized in [Table 3.1](#). In the following, we briefly introduce the two kernels that are widely employed in CAD: ACIS and Parasolid.

ACIS is an object-oriented C++ geometry library that integrates wireframe, surface, and solid modeling with both manifold and nonmanifold topology. It gives application developers a rich set of geometric operations for constructing and manipulating complex models. These include extruding,

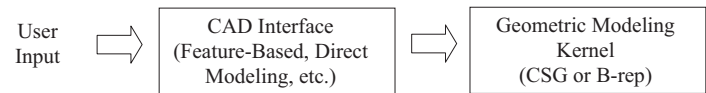


FIGURE 3.23 Relationship of a CAD Interface to Geometric Modeling Kernels.

Table 3.1 Geometric Modeling Kernels and CAD Systems

CAD Systems	Software Developer	Geometric Modeling Kernel
AutoCAD 2000	AutoDesk	ACIS
Pro/ENGINEER	PTC	Granite one
I-DEAS	SDRC	Geomod
Unigraphics	EDS	Parasolid
SolidWorks	Dassault systems	ACIS
CATIA	Dassault systems	CGM (Convergence geometric modeler)
NX	Siemens	Parasolid

sweeping, lofting, skinning, offsetting, slicing, stitching, sectioning, fitting, and interpolating surfaces. ACIS also offers a complete set of Boolean operations, and length, area, and mass property inquiry functions. The ACIS kernel outputs a SAT file format that any ACIS-enabled application can read directly.

Parasolid from Unigraphics Solutions is an exact B-rep modeler that supports solid modeling and integrated free-form surface and sheet modeling. Parasolid's Extreme Modeling is a set of tightly integrated, proprietary technologies that enable modeling of complex geometry. Parasolid comprises more than 600 object-oriented functions for applications running on Windows, UNIX, and LINUX.

3.4 Solid model build plan

One of the objectives in creating solid models for product design using the e-Design paradigm is to capture design intents so that design changes can be made by simply modifying a few dimension values and rebuilding the solid models. To achieve the objective, it is important for the designer to plan ahead and spell out detail steps in terms of the type of features, their sketch profiles, relations and dimensions, parent-child relationships, and the feature creation sequences, including the construction of datum features. This is especially true when we use a CAD system with a feature-based parametric modeling method, which is still the mainstream technology implemented in major CAD systems.

Why do we need to spend time in developing such a part build plan? First, we always want to complete our work in the minimum time with the best result. If we plan ahead and think through the best possible way to create a design in using a CAD system, we often foresee possible pitfalls and are able to take precautions, which save us time in the end by not throwing out the problematic and incomplete models and starting over. So, please think before you do it! Planning ahead saves you time and makes you a better CAD user.

Let us take a look at the two examples in [Figure 3.24](#). How should we construct the hose support and bracket shown in [Figures 3.24a](#) and [b](#), especially when a design change is anticipated for the bracket, as illustrated in [Figures 3.25a](#) and [b](#)? After all, we are not just creating a part, we are creating a quality part that is clean, organized, and well thought of.

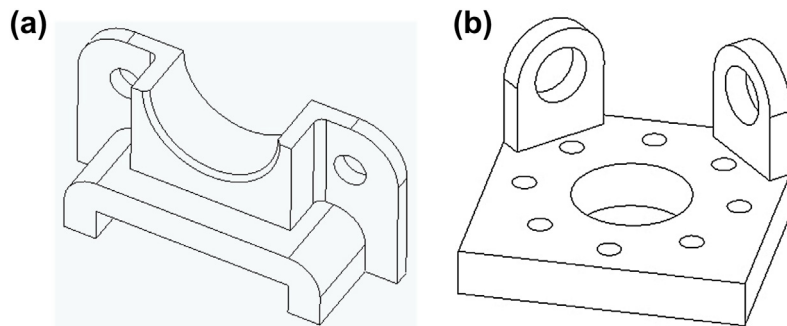


FIGURE 3.24 Sample Parts for Illustrating Part Build Plan: (a) Hose Support and (b) Bracket.

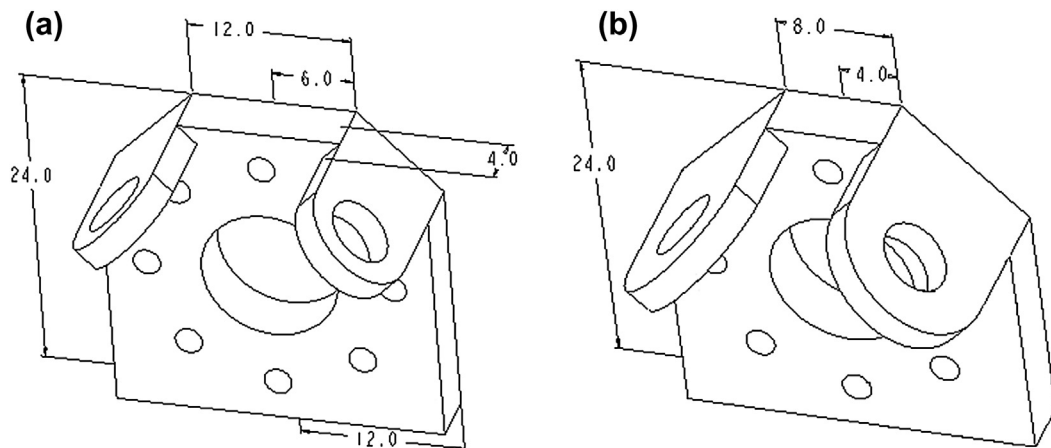


FIGURE 3.25 Design Intent for the bracket shown in Figure 3.24(b). (a) Dimension Design Variable. (b) Dimension Changed from 12 to 8.

What do we mean by a quality part? A quality part must be accurate in revealing geometric features in support of product design and manufacturing. Its feature construction sequence must be logical so that other team members can understand the solid model. The part should have minimum number of features. However, it does not imply that you must create complex sketches. It is a tradeoff in minimizing the number of geometric features and complexity level of individual sketch profiles. Moreover, a quality part must have minimum number of dimensions, implying as many sketch relations as possible. Most important, a quality model must be correctly parameterized and capture design intents. More about design parameterization is discussed in Chapter 5.

What should be included in the part construction plan? At least three things must be included:

1. Features and feature creation sequences (also include construction or datum features)
2. Sketch of each feature, including sketch plane, geometric entities, dimensions, and relations
3. Equations between dimensions as needed

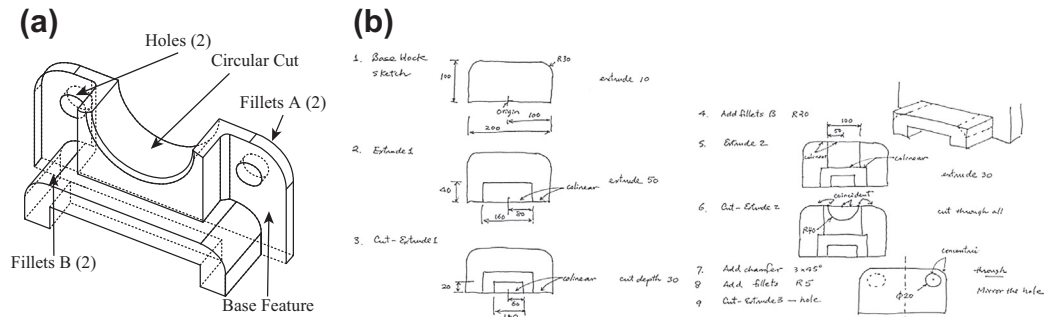


FIGURE 3.26 Build Plan for the hose support shown in Figure 3.24(a). (a) Major Features. (b) Hand-Sketched Sample Build Plan.

After a plan is jotted down on paper, it is a good idea to review it and try to optimize it before implementing it in CAD. The plan does not have to be fancy; it does have to facilitate your work. Note that a part construction plan is not unique. There is no “best plan”.

Let us take a look at sample the example part: hose support shown in Figure 3.24a. Usually the first question to come to mind is how many solid features need to be created and which one the base feature (first solid feature) should be. For this part, it seems to be logical to create the back plate as the base feature (see Figure 3.26a). Is it a good idea to create the semicircle in the sketch of the base block? Is it a good idea to create fillets A in the sketch? How about the holes? Is it a good idea to take the advantage of part symmetry by focusing on only half of the part and then mirror the first half for the remaining half? Again, there is no “correct” answer to these questions. The key principle is creating a quality part with a minimum effort. A hand-sketched sample build plan for the hose support example is given in Figure 3.26b for your reference.

Now, let us discuss the bracket example. The detailed dimensions are provided in Figures 3.27a and b.

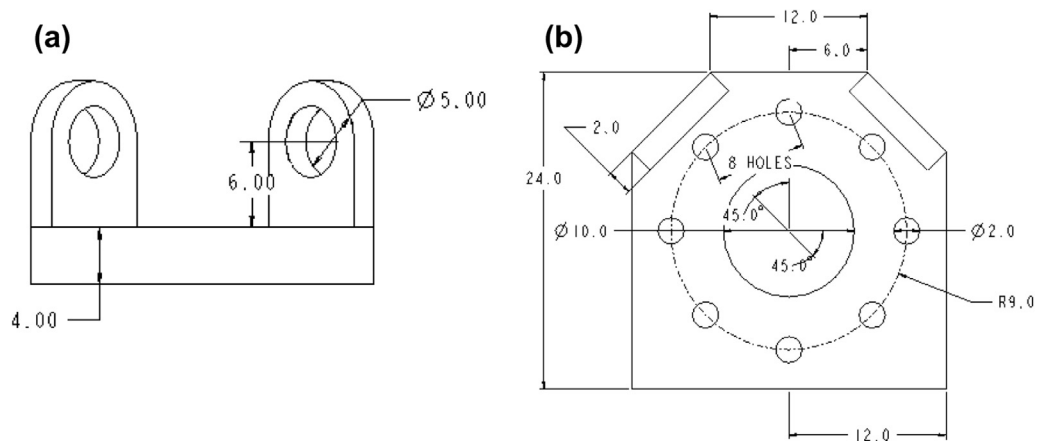


FIGURE 3.27 Sample part: bracket with Dimensions. (a) Front View. (b) Top View.

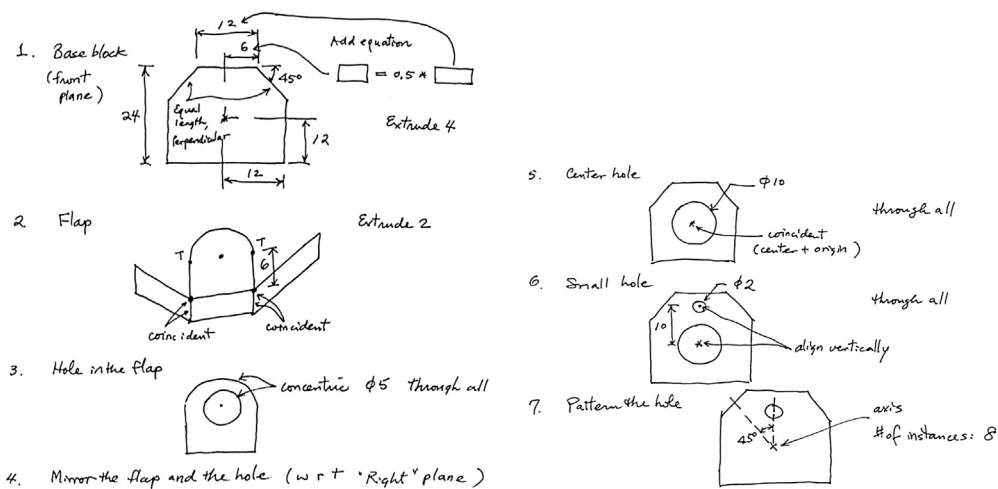


FIGURE 3.28 Hand-Sketched Build Plan for Sample Part: Bracket.

Note that the design variable is the length dimension of the top edge (current value: 12.0, shown in Figure 3.27b). When the design variable is changed, we expect to see the changes shown in Figure 3.25b; that is:

1. The 45° edges of the base block remained 45°, but with their lengths changed in accordance with new design variable value.
2. The back faces of the flaps align with the 45° edge of the base block.
3. The hole in the flaps stays in the middle with size unchanged.

How do we create such a part with the requirements described above? A sample build plan is shown in Figure 3.28 for your reference. To a less experienced CAD user, for a part like the bracket shown in Figure 3.24b, if you do not think ahead, you may end up with throwing away several “wrong” models before actually creating one that works. So please think before you actually construct a solid model in front of a computer.

3.5 Commercial CAD systems

Since the early 1960s when the first wireframe computer graphics was invented at MIT’s Lincoln Laboratory, CAD has advanced significantly and has become the de facto design tool for the industry around the world. The first commercial applications of CAD were in large companies of automotive and aerospace industries, as well as in electronics. Only large corporations could afford the computers capable of performing the calculations. Notable company projects were at GM with DAC-1 (Design Augmented by Computer) in 1964 and at Renault–UNISURF 1971 car body design and tooling.

As computers became more affordable, the application areas have gradually expanded. The development of CAD software for personal desktop computers was the impetus for almost universal application in all areas of engineering. The most significant development appeared in the mid-1990s,

in which major CAD tools were made available in PCs that allowed end users in mid- and small-size companies to be able to bring design from drawing board into digital form.

3.5.1 General purpose codes

Several general purpose commercial CAD systems available today were developed decades ago. These systems include the solid modeling packages Romulus (ShapeData) and Uni-Solid (Unigraphics), and the release of the surface modeler CATIA (Dassault Systemes) in 1981. Autodesk was founded in 1982, which led to the 2D system AutoCAD. Integrated Design and Engineering Analysis Software (I-DEAS) was produced by Structural Dynamics Research Corporation in 1982, and used primarily in the automotive industry, most notably by Ford Motor Company and General Motors. The next milestone was the release of Pro/ENGINEER in 1988, which heralded greater usage of feature-based modeling methods and parametric linking of the parameters of features. Also of importance to the development of CAD was the development of the B-rep solid modeling kernels Parasolid (ShapeData) and ACIS (Spatial Technology Inc.) at the end of the 1980s and beginning of the 1990s. This led to the release of mid-range packages such as SolidWorks in 1995, Solid Edge (then Intergraph) in 1996, and Autodesk Inventor in 1999. Several major mergers occurred throughout the years. SDRC was bought in 2001 by Electronic Data Systems, which had also acquired UGS Co. (maker of Unigraphics). EDS merged these two products into NX. UGS was purchased by Siemens AG in 2007 and was renamed Siemens PLM Software.

All major CAD systems offer not only solid modeling capabilities, but also CAE and CAM. Some CAD systems are equipped with in-house CAE/CAM, such as CATIA and Pro/ENGINEER. Some CAD partners with third-party software developers and fully integrates the third-party codes to the system; for example, CAMWorks integrated with SolidWorks. Although all major CAD systems offer excellent solid modeling and CAE/CAM, they serve different industrial sectors with slightly different focuses. CATIA is widely used by aerospace and automotive industry because of its superior surface modeling capabilities. AutoDesk is popular in small and mid-size companies due to its excellent capability in 2D drafting and its availability on PC in early years. Pro/ENGINEER serves heavy equipment industry, such as Caterpillar, due to its pioneer parametric modeling technology and strong CAE in the 1990s. SolidWorks became popular in almost all industrial sectors, as well as academia, because the software is intuitive and easy to use.

Several review articles on CAD software tools, such as those offered by 10 Top Ten Reviews (cad-software-review.toptenreviews.com), Cadalyst (www.cadalyst.com/listing/9/3d-modeling), and Wikipedia (http://en.wikipedia.org/wiki/List_of_computer-aided_design_editors, and http://en.wikipedia.org/wiki/Comparison_of_3D_computer_graphics_software), provide in-depth reviews and comparisons among major commercial systems. Readers are strongly encouraged to take a look at these articles for a better understanding of commercial CAD software.

3.5.2 Special codes

Besides general-purpose CAD software tools, there are at least two special codes worth mentioning. They are Rhinoceros (www.rhino3d.com) and SpaceClaim Engineer (www.spaceclaim.com/en/default.aspx).

Rhinoceros (Rhino) is a stand-alone, commercial nonuniform rational B-spline (NURB)-based 3D modeling software, commonly used for industrial design, architecture, marine design, jewelry design, automotive design, as well as the multimedia and graphic design industries. Rhino specializes in

free-form NURB modeling. Rhino is gaining popularity due to its diversity, multidisciplinary functions, low learning curve, relatively low cost, and its ability to import and export many file formats, which allows Rhino to act as a “converter” tool between programs in a design workflow.

SpaceClaim Engineer is a 3D direct modeler. It enables engineers to easily create concepts and prepare 3D designs for prototyping, analysis, and manufacturing without becoming experts in traditional feature-based CAD systems. SpaceClaim helps engineers interact with CAD geometry in new ways. Without becoming a CAD expert, users can edit models, conceptualize on-the-fly, and communicate quickly and easily with prototyping and manufacturing. Direct modeling changes the way designers think about working with 3D solid models by letting them focus on what they are designing. Intuitive tools such as Pull and Move let users directly select portions of the model and move them where users want. The Combine tool slices and divides parts into pieces and lets users merge in portions from other designs. The Fill tool cleans up small features and fills holes. Together, these direct modeling tools let designers get job done without resorting to traditional CAD.

3.6 Summary

There is no doubt that CAD offers a better visualization of the design, easier creation of drawings once the model is completed, and better integration with CAE and CAM for product development. We discussed in this chapter the fundamentals in solid modeling, including CSG and B-rep, the two most commonly employed methods for underline solid modeling in CAD. We introduced the mainstream solid modeling technique—feature-based parametric solid modeling—that is employed in major CAD systems. We discussed key concept and theories, including variational and parametric modeling techniques, the parent–child relationship, and feature construction sequence or history tree. We walked through the steps of model rebuild in CAD using a simple example. We also briefly introduced geometric modeling kernels and newly developed direct modeling method. In addition, we offered a brief overview of commercial CAD systems. We hope you have gained adequate knowledge of CAD and solid modeling techniques and understand the behind-the-scenes operations that CAD carries out when you interact with it.

It is important to point out that although CAD becomes essential for product design, especially using the e-Design paradigm, it has a few issues. First, CAD can be slow for conceptual design. In the early stages, we tend to think faster than anybody could model in 3D. The direct modeling method may offer good alternatives to this issue. Also, CAD may require a lot of computing power to handle complex parts and assemblies. Display and rendering such models can be slow and model rebuild due to design changes can be too sophisticated to handle. Finally, model interchange between 3D parametric CAD systems is still an open issue. More about CAD interoperability is discussed in Chapter 6. We are now ready to move on to the next chapter to discuss the theory and methods employed by CAD for assembly. Our goal again is to understand the behind-the-scenes operations in CAD when we use it for creating assemblies.

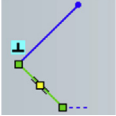
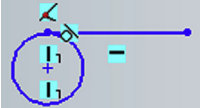
Appendix A Sketch relations

Sketch relations play an important role in solid modeling. In this appendix, we offer tables that illustrate the commonly seen relations (also called sketch constraints) in SolidWorks and Pro/ENGINEER in [Table A.1](#). Examples of such relations in SolidWorks are provided in [Table A.2](#).

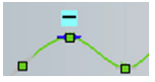
Table A.1 Commonly Seen Sketch Relations (or Constraints) in *Pro/ENGINEER* and *SolidWorks*

Pro/E	SolidWorks	Name	Entities	Descriptions
H or V		Horizontal or vertical	One or more lines or two or more points	The lines become horizontal or vertical.
		Alignment	Two or more vertices	The items are aligned vertically or horizontally.
$\perp$		Perpendicular	Two lines	The two items are perpendicular to each other.
//		Parallel	Two or more lines	The items remain parallel.
T		Tangent	An arc, ellipse, or spline, and a line, or arc	The two items remain tangent.
$\oplus$		Concentric	Two or more arcs, or a point and an arc	The circles and/or arcs share the same center point.
		Coincident	A point and a line, arc, or ellipse	The point lies on the line, arc, or ellipse.
L1 L1 R1 R1		Equal	Two or more lines, or two or more arcs	The line lengths or radii remain equal.
$\rightarrow \leftarrow$	N/A	Symmetric	A centerline and two points, lines, arcs, or ellipses	The items remain equidistant from the centerline, on a line perpendicular to the centerline.

Table A.2 Examples of Sketch Relations Seen in *SolidWorks*

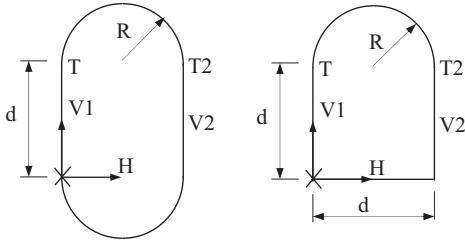
Relations	Icons	Notes
Horizontal		Horizontal line sketched.
Perpendicular		Second line was sketched perpendicular to the first. Sketch tool is active, so midpoint sketch snap is displayed on line.
Parallel		Two lines sketched with parallel relation.
Horizontal and tangent		Tangent arc added to horizontal line.
Horizontal and coincident		Second circle. Sketch tool is active, so quadrant sketch snaps display on the second arc.
Vertical, horizontal, intersection, and tangent		Circle sketched with center inferred to sketch origin (vertical). Horizontal line intersects circle quadrant. Tangent relation added.

Continued

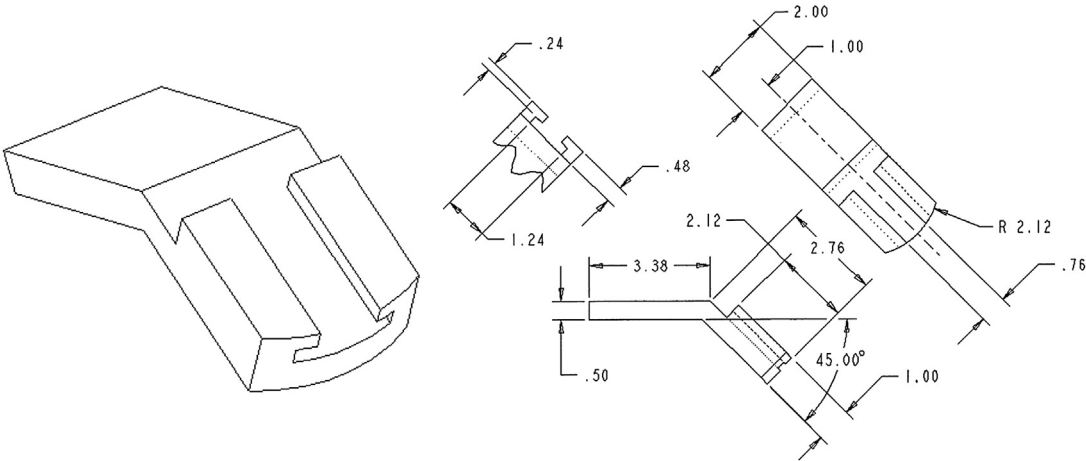
Table A.2 Examples of Sketch Relations Seen in <i>SolidWorks</i> —cont'd		
Relations	Icons	Notes
Horizontal, vertical, and equal		Horizontal and vertical relations inferred. Equal relation added.
Concentric		Concentric relation added.
Horizontal		Horizontal relation added to spline handles.

Questions and exercises

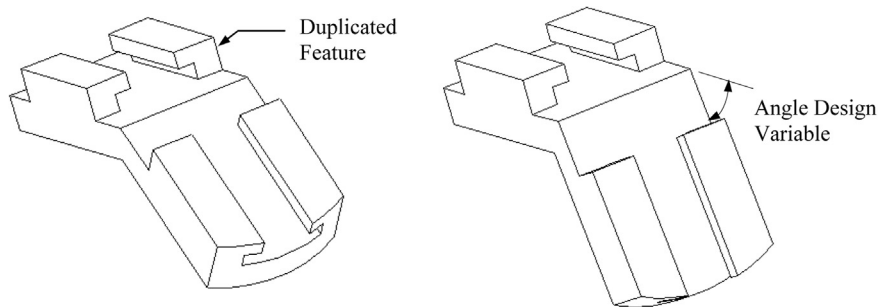
1. Are the following sketches *fully defined*? Why or why not? Please answer the question by formulating equations similar to those discussed in [Section 3.3](#).



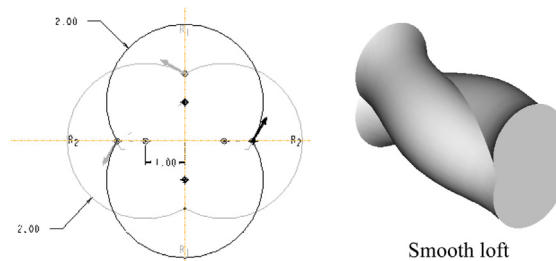
2. Create a solid model shown below. Make sure your model has identical dimensions as shown.



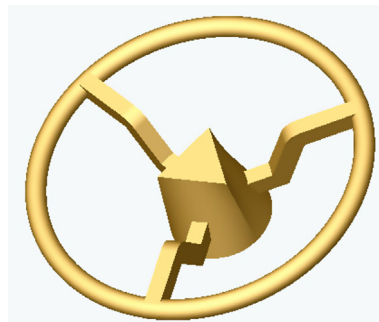
- a. Duplicate the extrusion feature to the top flat surface of the part as shown below (left).



- b. Define necessary relations so that when the angle design variable changes, the copied feature stays on the surface (above, right).
- c. Submit four views, front, top, side, and isometric of the final part (including hidden lines; angle design variable = 45°). Submit a screen capture that shows equation employed to capture the angle design variable.
3. Create a solid model with a smooth loft feature using the exact dimensions shown in the sketch. Note that there are three sections in the loft, and the distances between them are 5 units for each pair. Note that you will have to state how guide curves if any are defined to create such a loft.

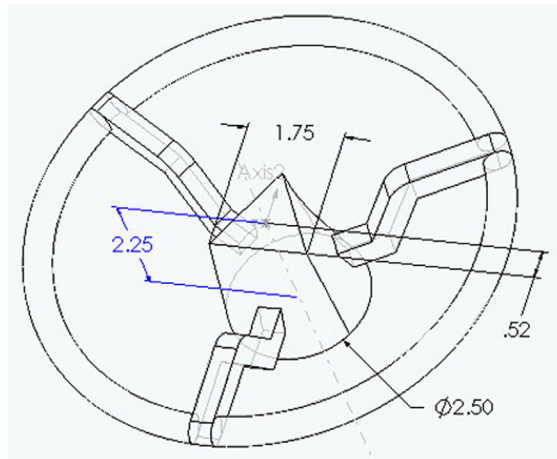
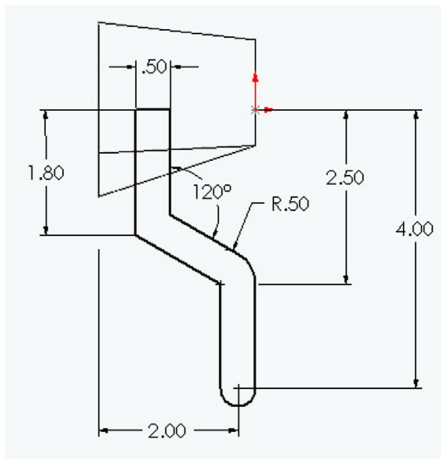


- a. Define relation(s) so that modifying a single dimension can change the diameter of all arcs. Note that the relation(s) must be implemented in the solid model and you must show screen captures of the solid model before and after changes, with a single dimension change.
- b. What is the range of the diameter dimension that you can change that results in valid solid models?
4. Create a steering wheel solid model using the exact dimensions shown in the figures below.



Note that the width of the spoke is 0.4; the triangle on top of the hub is an equal-lateral triangle. The bottom sketch profile of the hub is a circle with diameter 2.5. The distance between the bottom edge of the triangle and the origin is 0.52, and the height of the hub is 2.25 as shown below.

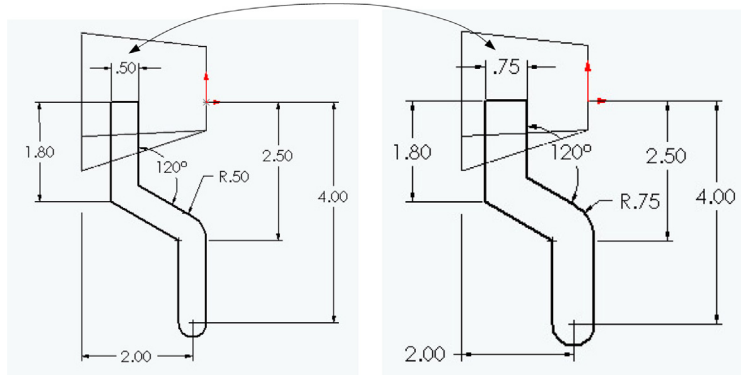
- a. What is the outer diameter of the wheel? _____



- b. Define relations and equation(s) so that when the height dimension of the spoke is changed—for example, from 0.5 to 0.75—the sketch profile of the spoke can be regenerated as shown.
 Relations: Please show relations on the sketch with brief explanation.
 Equation(s): Please list all equations and point the dimensions involved in the sketch or part.
 What is the range of the outer diameter of the wheel that you can change that results in valid solid models?

Minimum = _____, Maximum: _____

Height of the Spoke



- c. Please submit four views, front, top, side, and isometric of the final part (including hidden lines).

References

- Bozdoc, M., 2003. The History of CAD. www.mbinfo.mbdesign.net/CAD-History.htm.
- Lee, W., 1999. Principles of CAD/CAM/CAE Systems. Addison Wesley Longman, Inc.
- Planchard, D.C., Planchard, M.P., 2013. Official Certified SolidWorks Professional (CSWP) Certification Guide with Video Instruction. SDC Publications.
- Pratt, M.J., Wilson, P.R., 1985. Requirements for Support of Form Features in a Solid Modeling System. CAM-I, R-85-ASPP-01.
- Toogood, R., Zecher, J., 2012. Creo Parametric 1.0 Tutorial and MultiMedia DVD. SDC Publications.